

The effects of data quality on machine learning performance on tabular data

Sedir Mohammed^a , Lukas Budach^a, Moritz Feuerpfeil^a, Nina Ihde^a, Andrea Nathansen^a, Nele Noack^a, Hendrik Patzlaff^a, Felix Naumann^a , Hazar Harmouch^b

^a Hasso Plattner Institute, University of Potsdam, Germany

^b University of Amsterdam, The Netherlands

ARTICLE INFO

Dataset link: <https://github.com/HPI-Informatik-Systems/DQ4AI>

Keywords:

Data errors
Data-centric AI
Data pollution
Explainability
Machine learning

ABSTRACT

Modern artificial intelligence (AI) applications require large quantities of training and test data. This need creates critical challenges not only concerning the availability of such data, but also **regarding its quality**. For example, **incomplete, erroneous, or inappropriate training data can lead to unreliable models that produce ultimately poor decisions**. Trustworthy AI applications require **high-quality training** and test data along many quality dimensions, such as accuracy, completeness, and consistency.

We explore empirically the relationship between six data quality dimensions and the performance of 19 popular machine learning algorithms covering the tasks of classification, regression, and clustering, with the goal of explaining their performance in terms of data quality. Our experiments distinguish three scenarios based on the AI pipeline steps that were fed with polluted data: polluted training data, test data, or both. We conclude the paper with an extensive discussion of our observations.

1. Data quality and AI

The rapid advances in the field of artificial intelligence (AI) represent a great opportunity for further enhancement for many industries and sectors, some of which are critical in nature, such as autonomous driving and medical diagnosis. The potential for AI has been enhanced by the recent and future enormous growth of data. However, this precious data raises tedious challenges, such as data quality assessment, and, according to [1] data management, data democratization and data provenance.

Until recently, both academia and industry were mainly engaged in improving or introducing new and improving existing machine learning (ML) models, rather than finding remedies for any data challenges that fall beyond trivial cleaning or preparation steps. Nevertheless, the performance of AI-enhanced systems in practice is proven to be bounded by the quality of the underlying training data [2]. Moreover, data have a long lifetime and their use is usually not limited to a specific task, but can continuously be fed into the development of new models to solve new tasks. These observations led to a shift in research focus from a model-centric to a data-centric approach for building AI systems [3]. In 2021, two workshops emerged to discuss the potential of data-centric AI and to initiate an interdisciplinary field that needs expertise from both data management and ML communities.¹

In the field of data management, data quality is a well-studied topic that has been a major concern of organizations for decades, leading to the introduction of standards and quality frameworks [4,5]. The recent advances in AI have brought data quality back into the spotlight in the context of building “data ecosystems” that cope with emerging data challenges posed by AI-based systems in enterprises [1]. Researchers pointed out such challenges, including data quality issues [6], data life cycle concerns [7], the connection to ML-OPs [8], and model management [9]. Furthermore, some studies presented a vision of data quality assessment tools [10], an automation of data quality verification [11] or a methodology to summarize the quality of a dataset as datasheets [12], nutritional labels [13], and data cards [14].

In this work, under the umbrella of data-centric AI, we revisit six selected data quality dimensions, namely *consistent representation*, *completeness*, *feature accuracy*, *target accuracy*, *uniqueness* and *target class balance*. Our ultimate aim is to observe and understand ML model behavior in terms of data quality. We test a variety of commonly used ML algorithms for solving classification, clustering, or regression tasks. We analyze the performance of 19 ML algorithms covering the spectrum from simple models to complex deep learning models.

Before diving further into the details of our experimental setup, we highlight the broad scope of plausible variations of such an empirical

* Corresponding author.

E-mail addresses: sedir.mohammed@hpi.de (S. Mohammed), lukas.budach@student.hpi.de (L. Budach), Moritz.Feuerpfeil@student.hpi.de (M. Feuerpfeil), nina.ihde@student.hpi.de (N. Ihde), andrea.nathansen@student.hpi.de (A. Nathansen), nele.noack@student.hpi.de (N. Noack), hendrik.patzlaff@student.hpi.de (H. Patzlaff), felix.naumann@hpi.de (F. Naumann), h.harmouch@uva.nl (H. Harmouch).

¹ <https://datacentricai.org/neurips21/> and <https://hai.stanford.edu/events/data-centric-ai-virtual-workshop>

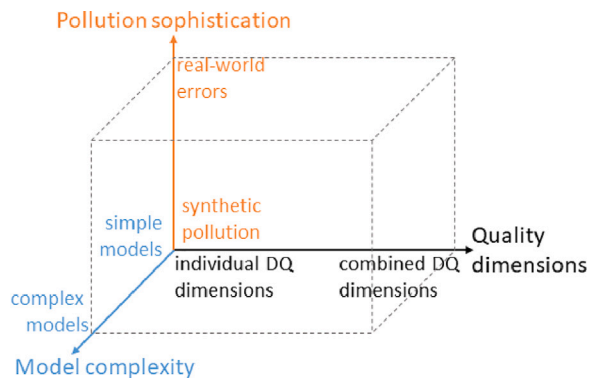


Fig. 1. The wide scope of empirically studying the effect of data quality on ML algorithm performance.

study, showing the full perspective of any possible correlation between data quality and ML models. As illustrated in Fig. 1, there are three main aspects: (1) the model that can vary from simple models (e.g., decision trees) to complex ones (e.g., based on pre-trained embeddings); (2) the pollution/error type can range from synthetically introduced problems to more difficult-to-detect real-world errors; (3) data quality dimensions could be studied individually or by considering several dimensions at once assuming they are not independent. Together, these dimensions span an enormous experimental space. To gain the necessary basic understanding, we limit our experiments to full range of traditional and deep ML models, synthetic pollution, and individual data quality dimensions. We plan to expand our study to cover the wider variations in future work.

Regarding data quality, we account for two aspects. First, data plays a different role at different stages of the ML pipeline: Some systems use pre-trained models and thus the only available data is the “test” data; in many other cases, data scientists also need “training” data to build the models from scratch. Second, training and testing data can be generated or collected by the same process from the same data source, so that they have similar quality. In a more realistic case, training and testing data have different quality, especially when using pre-trained models and different sources or collection processes. To that end, we consider in this study three scenarios: Training and testing data have the same quality (Scenario 3); the training data have high quality (in terms of the studied quality dimensions) and lower quality testing data (Scenario 2); and finally, the testing data have a high quality and the data used to build the models are of a lower quality (Scenario 1).

To vary data quality in each of these scenarios, we apply *data pollution* or corruption to create degraded quality versions of the dataset at hand. For each of the six quality dimensions, we designed a parameterized data polluter to introduce corresponding data errors. While we used real-world data only, for several of the datasets we had to manually create a clean version as a baseline to initiate the pollution process. In these cases, we report the performance of ML models for both the “original” and the “baseline” datasets.

Research in the ML community has studied the effects of label noise and missing values, and the data management community has studied the effects of data cleaning on classification, as we discuss in Section 2. Nevertheless, this paper is the first systematic study of the effects of data quality dimensions not only for classification, but also for clustering and regression tasks, while also considering various ML algorithms and different scenarios with training and test data of different data quality. Our work on real-world datasets with numerous experiments is a first step not only towards linking ML model performance to the underlying data quality, but also to understand and explain their connection.

Contributions. We present a comprehensive experimental study to understand the relation between data quality and ML model performance under the umbrella of data-centric AI, providing:

- A systematic empirical study that investigates the relation between six data quality dimensions and the performance of 19 ML algorithms.
- A simulation of real-life scenarios concerning data in ML pipelines. We perform a targeted analysis for cases where serving data, training data, test data, or both are of low quality.
- Practical insights and learned lessons for data scientists. In addition, we raise several questions and point out possible directions for further research.
- The open-source polluters, ML pipelines and all datasets as research artifacts are easily extendable with further quality dimensions, models, or datasets.

Outline. Next, we discuss related work in Section 2. Then, we formally define the six data quality dimensions together with a systematic pollution method for each in Section 3. In Section 4, we briefly introduce the 15 ML algorithms for the three AI tasks of classification, regression, and clustering. We describe our experimental setup in Section 5. The results of the empirical evaluation, the core contribution of this paper, are discussed in Section 6. Finally, we summarize our findings in Section 7.

2. Related work

First, we report on the state of the art in data validation for ML. Then, we discuss related work that studies the influence of data quality on ML models, namely by conducting an empirical evaluation, cleaning the data or by focusing on a specific error type like label noise. We summarize related work in Table 1, highlighting the respective considered data quality dimensions, employed ML algorithms, and methodologies.

Data validation. Several approaches have emerged to validate ML pipelines as well as the data fed to them, which includes training and serving data (data used in production). These approaches use the concept of *unit tests* to help engineers diagnose model-quality issues originating from data errors. For instance, the validation system implemented by Breck et al. [2] and the similar system by Schelter et al. [20] focus on validating serving data given a classification pipeline as a black box.

Generally, validation systems check, on the one hand, for traditional data quality dimensions, such as consistency and completeness, and on the other hand for ML dependent dimensions, such as model robustness and privacy [21]. To help data scientists with the validation task, Schelter et al. introduced the experimental library JENGA [22]. It enables testing ML model’s robustness under data errors in serving data. The authors use the concept of polluters or data corruptions as in our work. However, they do not provide an extensive experimental study and their focus is on describing the framework.

Task-dependent data quality. Foroni et al. argue that data quality assessment should not be performed in isolation from the task at hand [16]. Our results confirm this statement for ML models, as the same “low” quality data has a different effect when used to train different models. Their paper proposes a theoretical framework with a setup similar to our experiments, which evaluates the performance of a task given polluted datasets by various kinds of generated systematic noise. The authors considered a multitude of errors in their work, which we can map to the data quality dimensions of our study as follows: completeness (missing values, generation of nulls), consistent representation (synonyms, abbreviations), and feature accuracy (spelling mistakes, permuted words, terms in different languages, numerical variations, scale modifications, arithmetic negations). The proposed framework then computes the variation effect factor or the sensitivity factor from the observed results of the task. Unlike our work, however, the authors focused only on polluted training and testing data (Scenario 3 in our paper) and the experiments were conducted on a single dataset to evaluate only three models (one per ML task) namely, Random Forest, *k*-means, and Linear Least Square. The authors made observations that agree with our findings, especially the fact that missing values (completeness) are a problem for all ML tasks.

Table 1
Overview of related work.

Reference	Data Quality Dimensions	ML Algorithms	Methodologies
CleanML [15]	completeness, feature accuracy, uniqueness, consistent representation, target accuracy	Logistic Regression, KNN, Decision Tree, Random Forest, AdaBoost, Naive Bayes, XGBoost	Considered different scenarios of dirty training and test data.
Foroni et al. [16]	completeness, consistent representation, feature accuracy	Random Forest (classification), Linear Least Square (regression), k-means (clustering)	Introduces more errors to derive influence of low data quality.
Neutatz et al. [17]	uniqueness, consistent representation, feature accuracy, completeness	AutoSklearn builds model ensembles	Evaluation of state-of-the-art AutoML systems.
Shah et al. [18]	uniqueness	Random Forest, Logistic Regression, Neural Network, XGBoost	Analyzed the effect of duplicates.
Fréna and Verleysen [19]	target accuracy	Referenced papers using, among others, KNN, Bayes Decision Tree, SVM, AdaBoost, Decision Tree	Survey about the effect of dirty labels in training data.

Data cleaning. Li et al. investigated the impact of cleaning training data, i.e., improving its quality, on the performance of classification algorithms [15]. They obtained a clean version of the training data instead of systematically polluting it, as we did in this work. Their effort yielded the CleanML benchmark. They focused on five error types: missing values, outliers, duplicates, inconsistencies, and mislabels. These error types are among the most popular error types, and thus some correspond to the data quality dimensions in our study. The authors observed that cleaning inconsistencies and duplicates is more likely to have low impact, while removing missing values and fixing mislabeled data is more likely to improve the classifier prediction. These observations align with our findings.

The CleanML benchmark datasets have been recently used, among others, by Neutatz et al. [17]. The authors evaluate the ability of AutoML systems, such as AutoSklearn [23], to produce a binary classification pipeline that can overcome the effect of the following types of errors in training data, which we again map to our considered data quality dimensions: uniqueness (duplicates), consistent representation (inconsistencies), feature accuracy (outliers), completeness (missing values), and target accuracy (mislabels). The authors concluded that AutoML can handle duplicates, inconsistencies, and outliers, but not missing values. The paper also points out that most current benchmark datasets contain only few real-world errors with insignificant impact on the ML performance even without any cleaning. For this reason and as mentioned in Section 1, our work uses synthetic errors to better characterize the correlation between ML models performance and data quality.

Clearly, our study aligns with data cleaning efforts but with a different goal. However, our observations give data experts the understanding of the effects of data quality issues. They can then use this knowledge to determine the robustness of their insights and decide which specific problems should be tended to and when. For an overview of the progress in cleaning for ML, we refer to [24].

The work of Shah et al. follows a similar direction as CleanML and ours [18]. However, the authors focus solely on investigating the influence of deduplication, which we can map to the data quality dimension uniqueness in our work, on an underlying classification task. They consider five different ML algorithms: random forest, logistic regression, a neural network, and XGBoost. They also investigate the influence of different encodings on the underlying ML task. The authors used 16 real-world datasets for the experiments, in which they manually annotated duplicates. In addition to the complete removal of duplicates, the authors also look at the influence of a gradual introduction of duplicates into the data on the underlying ML task, similar to our setup. In general, the authors have similar findings to ours that certain ML algorithms, such as logistic regression, are more robust against duplicates than others.

Label noise. The problem of label noise or mislabeling is one of the main concerns of the ML-community and has attracted much interest [19]. This problem is in essence a data quality problem. Fréna and Verleysen surveyed the literature related to classification using training data that suffers from *label noise*, which is equivalent to the target accuracy dimension in our work [19]. They distinguish several sources of noise, discuss the potential ramifications, and categorize the methods into the classes “noise-robust”, “noise cleansing”, and “noise-tolerant”. They conclude that label noise has diverse ramifications, including degrading classification accuracy, high complexity learning models, and difficulty in specifying relevant features.

In summary, we present the first systematic empirical study on how both training and test data quality affects not only classification but all three ML tasks. We also provide a clear definition for each of the data quality dimensions and a respective method to systematically pollute the data.

3. Data quality dimensions and data pollution

We present the definition of the six selected data quality dimensions, along with our methods to systemically pollute a dataset along those dimensions. In this work, we use the ML terms *feature* and *sample* to refer to columns and rows, respectively. During pollution, we assume that features’ data types and the placeholders that represent missing values in each feature are given.

3.1. Consistent representation

A dataset is consistent in its representation if no feature has two or more unique values that are semantically equivalent. I.e., each real-world entity or concept is referred to by only one representation. For example, in a feature “city”, New York shall not be also represented as NYC or NY. Consistent representation is different from uniqueness, which focuses on ensuring that no duplicate records exist in the dataset (see Section 3.5).

Definition 1. The degree of *inconsistency* of a feature c , denoted as $InCons(c)$, is the ratio of the minimum number of replacement operations required to transform it into a consistent state and the number of samples in the dataset.

This definition applies only to categorical features, i.e., strings or integers that encode categorical values, whereas numerical features and dates are considered to be consistent and their $InCons(c) = 0$.

Definition 2. We define the degree of *consistency* of a dataset d with f features as follows.

$$\text{Consistency}(d) = 1 - \frac{1}{f} \cdot \sum_{i \in 1, \dots, f} \text{InCons}(c_i) \quad (1)$$

Example. Consider a dataset d with a single categorical feature “City”, containing the ten distinct values [New York, NYC, Los Angeles, LA, San Francisco, SF, NY, Los Angeles, SF, San Francisco]. This “City” feature includes multiple representations of the same cities: New York is also represented as NYC and NY, Los Angeles as LA, and San Francisco as SF. To achieve consistency in this feature, NYC and NY could be replaced with New York, LA with Los Angeles, and SF with San Francisco. The minimum number of replacement operations required to achieve this consistency is 5. Thus, the inconsistency of the “City” feature is $\text{InCons}(\text{City}) = \frac{5}{10} = 0.5$ and the degree of consistency for the dataset is calculated as $\text{Consistency}(d) = 1 - \frac{1}{1} \cdot 0.5 = 0.5$.

Pollution. We have two inputs: First, the percentage of samples to be polluted λ_c , defined by a value between 0 and 1, and second, for each unique value v of a pollutable feature, the number of representations k_v for that value (including v itself). For each categorical feature, we choose randomly the samples to be polluted. Then, we generate $k_v - 1$ new representations for each unique value v of its values. The new representations of a string value are produced as new non-existing values by appending a trailing ascending number to the end of the value, whereas for integers, new integers are added after the maximum existing one. These sample’s entries at this feature are replaced by a randomly picked fresh representation of the original value.

When pollution introduces alternative representations in the data, it typically decreases the measured consistency according to Eq. (1). However, as the quality measure is based on the number of unique representations rather than their semantic integrity (as defined by the clean data). Thus, consistency can be achieved by aligning the correct values to match the current state of the data. If the data contains significant inconsistencies – indicating substantial pollution – fewer transformations may be required to align the correct values with the noisy ones than to correct the noisy values themselves. As a result, the calculated consistency, as per Eq. (1), may increase. For consistency-related results, we show prediction accuracy curves only up to the point where measured consistency begins to increase during pollution.

3.2. Completeness

The problem of missing values exists in many real-world datasets. Some of these values are actually missing, e.g., missing readings due to a failure in a sensor, while others are represented by a placeholder, such as “unknown” or “NaN”.

Definition 3. The *completeness* of a feature c is the ratio of the number of non-missing values and the total number of samples n in this dataset. The completeness of a dataset d is defined as

$$\text{Completeness}(d) = 1 - \frac{1}{f} \cdot \sum_{i \in 1, \dots, f} \text{missing}(c_i) \quad (2)$$

where $\text{missing}(c_i)$ denotes the ratio of the total number of missing values in the feature c_i and the number of samples in the dataset.

For ML, samples with a missing value for the target feature are usually removed from the dataset, as they cannot be used for training. Thus, we exclude the target feature while computing completeness.

Example. Consider a dataset d that has two features (excluding the target feature) and four samples. This means that there are eight cells in total for computing the completeness. If two values of each feature are missing in the dataset, we calculate $\text{Completeness}(d) = 1 - \frac{1}{2} \cdot \left(\frac{2}{4} + \frac{2}{4}\right) = 0.5$.

Pollution. We inject missing values “completely at random” [25] according to a specified pollution percentage λ_c for each feature. If there are already missing values in the feature, we account for those values and inject only the remaining number of missing values necessary to reach λ_c . A feature-specific placeholder is used to represent all missing values injected into this feature. The placeholder value does not carry information except that the value is missing. A typical placeholder is “Not a Number” (NaN). Many implementations of ML algorithms cannot handle NaN values. For this reason, we choose a representation as placeholders that can be used in computations, but lie outside the usual domain of a feature and are still distinguishable from the actual non-missing values of the feature. For example, -1 for “age” feature, or the string “empty” for “genre” categorical feature in a movie table. We manually selected the used placeholders, as this task requires some domain knowledge to determine suitable values.²

3.3. Feature accuracy

Feature accuracy reflects to which extent feature values in a given dataset equal their respective ground truth values.

Definition 4. The *feature accuracy* measures the deviation of a column’s values from their respective ground truth values. For a categorical feature c , we define the feature accuracy as follows.

$$cF\text{Acc}(c) = 1 - \frac{\text{mismatches}(c)}{n} \quad (3)$$

where $\text{mismatches}(c)$ denotes the number of values in the feature c that are different from the ground truth, and n is the number of samples in the dataset. For numerical features, we define feature accuracy as follows.

$$nF\text{Acc}(c) = 1 - \frac{\text{avg_dist}(c)}{\text{mean_gt}(c)} \quad (4)$$

$$\text{avg_dist}(c_i) = \frac{1}{n} \cdot \sum_{j=0}^{n-1} |gt_{i,j} - v_{i,j}| \quad (5)$$

where $\text{avg_dist}(c)$ is the average of the absolute distances between the ground truth and values in c (see Eq. (5)) and $\text{mean_gt}(c)$ is the mean of the ground truth values of c .

In Eq. (5), j is used as the index of a specific sample. Hence, $gt_{i,j}$ denotes the ground truth and $v_{i,j}$ the value of the sample with index j in feature i . In contrast to the categorical feature quality measure, the numerical measure can fall below 0 and has no defined lower bound.

The feature accuracy of an entire dataset consists of two metrics: The average feature accuracy of all categorical features $cF\text{Accuracy}$ and the average feature accuracy of all numerical features $nF\text{Accuracy}$. The numbers of categorical and numeric features are given by n_{cat} and n_{num} , respectively.

$$cF\text{Accuracy}(d) = \frac{1}{n_{cat}} \cdot \sum_{i=0}^{n_{cat}-1} cF\text{Acc}(c_i) \quad (6)$$

$$nF\text{Accuracy}(d) = \frac{1}{n_{num}} \cdot \sum_{i=0}^{n_{num}-1} nF\text{Acc}(c_i) \quad (7)$$

Example. Consider a dataset d with five samples and with the two features “City” (categorical) and “Temperature” (numerical). In this example “City” has one mismatch, similar to the example for consistent representation, LA should be Los Angeles. We would calculate the feature accuracy for “City”, the categorical feature, as $cF\text{Accuracy}(d) = \frac{1}{1} \cdot \left(1 - \frac{1}{5}\right) = 0.8$.

² See <https://github.com/HPI-Information-Systems/DQ4AI/blob/main/metadata.json> for detailed setting of the placeholders.

Pollution. The pollution is executed differently, depending on the feature type. For categorical features, the level of pollution λ_{fa} for a specific feature c determines the percentage of samples to be polluted. The randomly selected samples are polluted by exchanging the current category with a random, but different category from feature c ' domain.

For numeric features, we add normally distributed noise to all samples of the feature c : $noise(c) = X \cdot mean_gt(c)$ where X is a random sample drawn from the normal Gaussian distribution $N(\mu, \sigma^2)$ with $\mu = 0$ and $\sigma^2 = \lambda_{fa}$. The level of pollution λ_{fa} determines the standard deviation of the normal distribution and thus denotes how wide it is spread.

3.4. Target accuracy

For each sample in a dataset, the target feature contains either a class/label in classification tasks or a numeric value in regression tasks. There might be some incorrect labels due to human or machine error, e.g., a dog labeled as "wolf". The definition of the target accuracy of a dataset is equivalent to the definition of the feature accuracy of its target feature (see previous section). Nevertheless, the target feature is the most important feature because of its influence on prediction performance. Thus, it is beneficial to study its accuracy separately.

Example. As the target accuracy is calculated similarly to feature accuracy, see Section 3.3 for an example.

Pollution. Naturally, we used the same pollution method as for feature accuracy, based on the target type.

3.5. Uniqueness

Redundant data does not provide additional information to the ML model for the training process. Thus, deduplication is a common step in ML pipelines to avoid overfitting. Exact duplicate rows are, in general, easy to detect. Yet, this step is still expensive, especially for large datasets.

Definition 5. The *uniqueness* of a dataset d is the fraction of unique samples within the dataset. We normalize the value as follows.

$$Uniqueness(d) = \frac{unique_samples(d) - 1}{n - 1} \quad (8)$$

We subtract 1 from the denominator and numerator to allow a quality metric score of 0.

Example. Consider a dataset d with ten samples. In this example, let three of the ten samples appear as exact duplicates of the other rows. Thus, seven of the ten samples are unique and we calculate the uniqueness for this example as $Uniqueness(d) = \frac{7-1}{10-1} = \frac{2}{3}$.

Pollution. To pollute a dataset along the uniqueness dimension, we first remove all existing exact duplicates. Then, we add exact duplicates of randomly selected samples to the dataset. The number of the added duplicates is determined by the duplication factor ρ : for each class cl with n_{cl} samples, we add $dup_{cl} = (\rho - 1) \cdot n_{cl}$ duplicates to avoid changing the class balance (see Section 3.6). Thus, the size of the polluted dataset is $n \cdot \rho$ and its uniqueness is $1/\rho$. For each randomly selected sample from a class cl , we add x duplicates of this sample and then continue sampling and adding duplicates to reach dup_{cl} . We draw x from a pre-defined distribution: we apply uniform, normal, and Zipf distributions, in addition to adding a single duplicate of each selected sample.

3.6. Target class balance

Many ML algorithms assume a relatively equal number of samples per target class, i.e., a balanced dataset, to achieve satisfactory performance. Clustering or classification algorithms on top of an imbalanced dataset may fail at identifying structures or even miss smaller classes completely. For example, the k -Means algorithm suffers from the "uniform effect", i.e., it recognizes clusters of approximately uniform sizes even if they are not present in the input data [26].

Definition 6. Given a dataset d with m target classes $cl_1, \dots, cl_m$ of $n_{cl_1}, \dots, n_{cl_m}$ samples per class, respectively, and $\forall i, j : 1 \leq i < j \leq m \iff n_{cl_i} \leq n_{cl_j}$, the target class *imbalance* is defined as the sum of the pairwise differences between the number of samples per class:

$$ImBalance(d) = \frac{1}{2} \cdot \sum_{i,j \in 1, \dots, m} |n_{cl_i} - n_{cl_j}| \quad (9)$$

As the worst imbalance case, we assume a maximal imbalanced dataset that has $\lceil m/2 \rceil$ classes with n_{cmax} samples and the remaining classes have 0 samples, where n_{cmax} is the maximum number of samples that a class can have. The target class imbalance of such a dataset is $\epsilon = \lceil m/2 \rceil \cdot \lfloor m/2 \rfloor \cdot n_{cmax}$.

Definition 7. The target class *balance* of a dataset d is the deviation from its imbalance score, normalized by the imbalance score of the worst case.

$$Balance(d) = 1 - \frac{ImBalance(d)}{\epsilon} \quad (10)$$

If all classes in the dataset have the same number of samples, then $Balance(d)$ is maximal and equals 1. In contrast, $Balance(d)$ reaches its minimum ($\lim_{d \rightarrow w} Balance(d) = 0$) if the balance of the classes in the dataset approaches the hypothetical worst case.

Example. Consider a dataset d with the three target classes *expensive*, *moderate*, and *cheap*. Let the number of samples in each respective class be $n_{expensive} = 10$, $n_{moderate} = 20$, and $n_{cheap} = 30$. We calculate the target class imbalance using the sum of pairwise differences between the number of samples per class as $ImBalance(d) = \frac{1}{2} \cdot (|10 - 20| + |10 - 30| + |20 - 30|) = 20$.

To compute the target class balance, we first need the imbalance score for the worst case. In the worst case scenario, two of the three classes have $n_{cmax} = 30$ samples, and one class has 0 samples. Thus, the worst-case imbalance score is $\epsilon = \lceil 3/2 \rceil \cdot \lfloor 3/2 \rfloor \cdot n_{cmax} = 60$. Finally, we compute the target class balance as $Balance(d) = \frac{20}{60} = 1 - 0.333 = 0.667$.

Pollution. We have two inputs for pollution: The degree of imbalance λ_{cb} and the number of samples in the polluted version $\tilde{n}$. We can choose $\tilde{n}$ arbitrarily as a multiple of m or calculate it from the data as the number of samples from the original dataset, needed to produce the maximum pollution level. We use λ_{cb} to calculate the number of samples per class in the polluted version. The degree of imbalance λ_{cb} for any polluted dataset version satisfies: $\lambda_{cb} = \frac{ImBalance(\tilde{d})}{1/3 \cdot (m+1) \cdot \tilde{n}}$.

To pollute, we order the target classes by their sizes descending and for the classes of the same size, we use the class ID in ascending order. Using the defined order, we create a class imbalance with an equal difference Δ for each subsequent pair of classes. The used samples of each class are randomly selected, i.e., in the polluted dataset, the following holds: $\forall 1 < j \leq m : (\tilde{n}_{c_{j-1}} \leq \tilde{n}_{c_j}) \wedge (\tilde{n}_{c_j} - \tilde{n}_{c_{j-1}}) = \Delta$. To create a class imbalance, we calculate the class size of a balanced dataset $\frac{\tilde{n}}{m}$ with $\tilde{n}$ samples and m classes. Then, we iteratively add/remove samples from the classes based on their order: We remove samples from all classes that are at indices $\lfloor \frac{m}{2} \rfloor - 1$ and below, and add samples to all classes at indices above that, unless m is odd: then the size of the class at index $\lfloor \frac{m}{2} \rfloor$ stays constant.

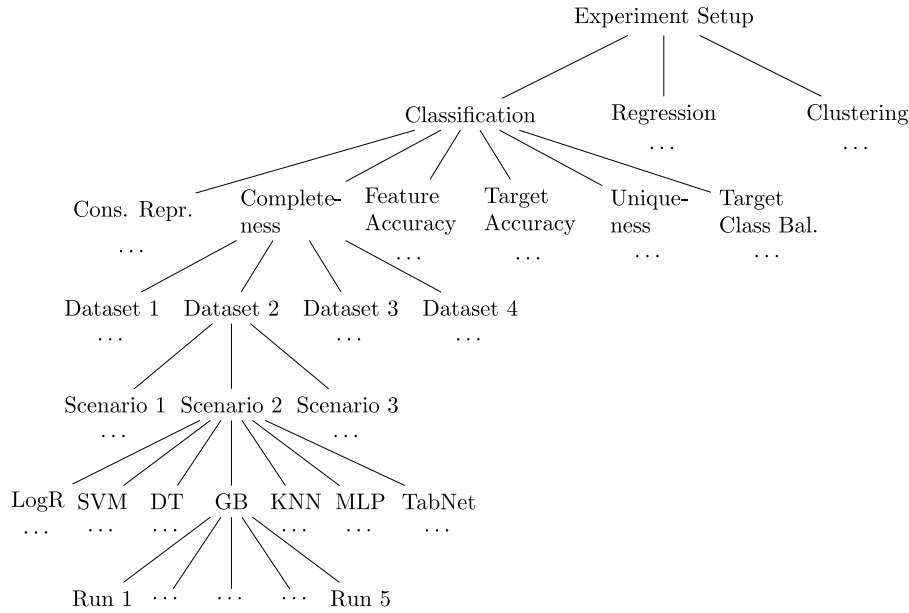


Fig. 2. Experimental setup yielding $(2 \times 6 \times 3 \times 3 \times 7 \times 5) + (1 \times 5 \times 3 \times 3 \times 7 \times 5) + (1 \times 6 \times 4 \times 1 \times 5 \times 5) = 4905$ individual experiments.

4. Machine learning tasks

This section introduces the 19 algorithms that we employed for the three machine learning (ML) tasks of *classification*, *regression* and *clustering*. Our selection was guided to cover a broad range of methods families for each task.

Classification. Classification is a supervised learning method that uses labeled data to learn the dependency between features and a discrete target variable. The goal is to assign samples to predefined classes (predict the correct class) by learning patterns in labeled training data. We selected a variety of classification algorithms that fall into different categories. We included two linear classification models: Logistic regression (LogR) [27] and support vector machine (SVM) [28]; two tree-based algorithms: a decision tree (DT) [29] and a gradient boosting (GB) [30] classifier; a k -nearest neighbors (KNN) classifier [31]; two neural network-based models: a multi-layer perceptron (MLP), in the form of three variants, concretely with 1 (MLP-1), 5 (MLP-5) and 10-hidden layers (MLP-10) [32] and TabNet [33], a transformer-based deep learning model.

Regression. Regression is a supervised learning method that uses labeled data to learn the dependency between features and a *continuous* target variable. There is a plethora of regression algorithms. To evaluate how error-prone different categories of regression algorithms are and how more complex algorithms within one category perform, we compare seven of the most widely used approaches for regression out of three categories of regression algorithms. We selected two linear-regression-based algorithms: a linear regression (LR) [34], and a ridge regression (RR) [35]; three tree-based algorithms: a decision tree (DT) [29], a random forest (RF) [36] and a gradient boosting (GB) regression; and again two deep-learning based approaches: a multi-layer perceptron (MLP) with 1 (MLP-1), 5 (MLP-5) and 10-hidden layers (MLP-10) [32] and TabNet [33] (TN).

Clustering. Clustering algorithms aim to find groups of samples with similar features in a given dataset. As clustering is an unsupervised task, our clustering models are not trained but directly receive all data as test data for the prediction of the target label. Depending on dataset properties, such as dimensionality, distribution or data types, current research recommends different clustering algorithms. These can be grouped into different categories or algorithm families. We decided to

use one algorithm from each of the five most commonly used categories of clustering algorithms [37]: the Gaussian mixture clustering algorithm [38] from the distribution-based family; the k -means [39] and the k -prototypes [40] algorithms from the centroid-based family; the agglomerative clustering algorithm [41] from the hierarchical family; the *ordering points to identify cluster structure* (OPTICS) algorithm [42] from the density-based family; and a deep autoencoder neural network from deep learning-based family [43].

5. Experimental setup

This section gives an overview of our implementation and introduces our datasets together with the parameterization and performance measures of the analyzed ML models. The experiments sweep a wide scope of parameters as shown in Fig. 2, including three tasks, six pollution types, four datasets, three pollution scenarios (for clustering just one scenario), seven approaches per task (five for clustering), and five runs to eliminate measurement variance (one for TabNet, to reduce extended training time). Multiplying, this yields 4905 experimental runs to produce all results.

5.1. Implementation

The implementation of the polluters and ML pipelines are written in Python 3.10 using scikit-learn 1.3.2 and PyTorch 2.1.2. We evaluate the performance of the ML models in three scenarios:

- Scenario 1 – polluted training set
- Scenario 2 – polluted test set
- Scenario 3 – polluted training and test sets

These scenarios are considered only for classification and regression, as clustering does not have a separate training and test set. To create the scenarios, we randomly split the data with a stratified 80:20 split into training and test sets.

We then pollute the training and test sets separately, varying the ratio of pollution between 0 and 1 in increments of 0.1. For the consistent representation dimension, we tested $k_v = 2$ and $k_v = 5$, thus adding one or four new representations per v in the polluted dataset, respectively. For uniqueness experiments, we varied ρ from 1 to 5 in steps to lead to linear quality decrease by 0.1 per step, i.e., $\frac{10}{9}$, $\frac{10}{8}$, ..., Before

Table 2
Overview of the used datasets after pre-processing.

Name	Samples	Features	Categorical	Numerical	Classes
Classification					
Credit	1 000	20	13	7	2
Contraceptive	1 473	9	7	2	3
Telco	7 032	19	16	3	2
COVID	1 025 152	17	16	1	2
Regression					
Houses	1 460	79	46	33	—
IMDB	5 993	12	8	4	—
Cars	15 157	8	3	5	—
COVID	1 025 152	17	17	0	—
Clustering (all datasets sampled)					
Bank	7 500	3	2	1	6
Coverttype	7 504	54	44	10	7
Letter	7 514	16	0	16	26
COVID	25 000	17	16	1	4

applying any ML model, we one-hot encode the categorical features. Then, we measure the performance of the respective ML models, given the specific scenario for the specific dataset polluted with the specific polluter configuration. We run each polluter configuration five times with a different random seed, obtaining five results per ML model in that setting, which we then aggregate by averaging.

For regression, we discretized the data with manually specified bin-step sizes before the stratified split. We use the discretized version also for the class balance and uniqueness polluters, as they require the target feature to consist of discrete classes. For all other polluters, we use the original continuous representation. The target feature distribution in our regression datasets is mostly close to a normal distribution.

Applying the class balance polluter would lead to a heavily decreased dataset size of much less than 50% after balancing. That is why, when using the class balance polluter, we discard the discretized classes with very few samples, which would result in a balanced dataset with such a small size. To allow consistent comparisons with the original dataset, i.e., having the same classes contained in the data, we also discard those classes with few samples from the original dataset for the experiments with the class balance polluter.

5.2. Datasets

To investigate the correlation between the studied data quality dimensions and the chosen ML algorithms, we use the ten datasets shown in Table 2. We chose them for a variety of domains, sample sizes and characteristics. Our choice was also influenced by the ML task that the dataset is used for.

Classification. IBM's Telco Customer Churn (Telco) dataset represents customers from a fictional telecommunications company [44]. The target variable Churn describes whether the customer cancelled their contract within the last month. The Credit dataset contains credits between the years 1973 and 1975 from a southern German bank [45]. The target variable tells whether a customer complied with the conditions of the contract or not. The Contraceptive dataset was part of the 1987 National Indonesia Contraceptive Prevalence Survey, asking non-pregnant married women about their contraceptive methods [46]. The classification task is to determine the contraceptive method choice. The COVID-19 Dataset (COVID) dataset, released by the Mexican government, contains anonymized data from patients either confirmed or suspected to be infected with the coronavirus [47,48]. The classification task is to predict patient mortality, i.e., whether a patient died during the hospital stay or afterwards (monitored by epidemiological surveillance units or health jurisdictions) [47].

Regression. The Houses dataset contains features of houses in the city to determine their sales price [49]. We use the dataset in the form it was presented in a Kaggle challenge [50]. The IMDB dataset contains features and ratings for films and series that were retrieved from the IMDB website [51]. The Cars dataset collects listings for used cars of different manufacturers [52]. The sales price is the feature that shall be predicted. The data is stored in files grouped by manufacturer. We use only the data of VW. We also used the COVID dataset, previously presented in the context of classification, for regression [47,48]. In this case, the goal is to predict the age of the patients.

Clustering. The Letter dataset contains statistical measures of character images [53,54]. The Bank dataset contains different characteristics of a person and has a binary target stating whether a term deposit is subscribed [55,56]. We use a subset of this dataset, taking the education level as the target and keeping only three features related to it to have more than two clusters. We removed all classes with fewer than 2 000 samples to avoid significant data loss when applying the target class balance polluter. The Coverttype dataset consists of descriptive information about forested areas [57,58]. We also used the COVID dataset in the context of the clustering task, where the objective is to cluster patients based on their COVID-19 test findings (three different degrees of severity or not infected) [47,48].

To evaluate clustering, we sampled all datasets as the last step of its preprocessing to reduce their size, as further processing would have required too much main memory. We slightly varied the number of samples selected to be a multiple of the dataset's class count larger than 7 500 (for COVID, 25 000). Instead of sampling once, we decided to also use the same five random seeds used for any random operations in the polluters to create a total of five preprocessed datasets for each dataset. This choice was made to account for the potential impact the sampling may have on the data if done using only one seed.

5.3. Model parameters

This section documents our parameter settings. All parameters that are not mentioned are kept at their default values in scikit-learn.

Classification. For LogR, we increase *max_iter* to 2 000 for better convergence. For a multi-class dataset, we fit a binary problem for each label. The maximum number of iterations for each MLP variant (MLP-1, MLP-5, MLP-10) is 1 000. For SVM, we use a linear kernel and scale the input data with the scikit-learn *StandardScaler*. For TabNet (TN), we set the maximum number of iterations to 10. Additionally we used a patience value of 3 to stop earlier — if the loss does not change further. For the COVID dataset, we used PyTorch for the experiments with MLP, SVM and KNN due to runtime considerations. In the case of MLP, we adjusted several hyperparameters, including reducing the maximum number of iterations to 10, as experiments showed no further change in loss beyond this point. Similar to TN, we introduced a patience value of three, along with a required loss rate change of 0.01. For KNN and SVM, we chose the default values of scikit-learn.

Regression. For LR and RF, we set *n_jobs* to -1 to use all available processors. For all MLP variants, we increase the *max_iter* parameter that defines the maximum number of epochs to 3 000. Similar to the classification task, we set the maximum number of iterations for TabNet to 10, along with a patience value of three. For the COVID dataset, we again used PyTorch for MLP and also set the maximum number of iterations to 10, with a patience value of 3.

Clustering. The actual number of clusters is passed to the *k*-Means/*k*-Prototypes, Gaussian mixture, and agglomerative algorithms. If categorical features exist, the agglomerative algorithm uses Gower's distance measure [59]. We set the *affinity* parameter to "precomputed" (a pre-calculated distance matrix is to be employed) and the *linkage* parameter to "average", which defines the distance between two clusters as the

average distance between all samples in the first and all samples in the second cluster. For OPTICS, we defined only the minimum cluster size parameter *min_cluster_size*, which specifies how many samples need to be in a cluster for it to be classified as such, as 100. Our autoencoder is trained for 200 epochs and optimized using the Adam optimizer [60] with a learning rate of 0.003 and mean squared error loss. The dataset is split into 80% training and 20% test data and loaded in batches of 128 shuffled samples. For all models that need a random seed, we used 42 as a seed.

5.4. Model performance

Classification. The most common performance metric in classification tasks is accuracy: the number of correct predictions over the number of total predictions. But accuracy can be misleading for imbalanced datasets. For example, a majority class prediction baseline yields a 90% accuracy on data that is naturally distributed among two classes with a ratio of 9:1. Therefore, we use the F_1 -score: it better accounts for class imbalance, which does exist in our datasets. For instance, the Telco and Credit datasets are unbalanced in their target classes, with a 70/30 split or worse. Usually, the F_1 -score is measured for a single target class, but as we do not make assumptions about the importance of the target class, we report the average of the F_1 -scores over all target classes.

Regression. Mean squared error (MSE) is the commonly used metric to evaluate regression algorithms. However, it is highly dependent on the data domain, e.g., it is expected to be much larger for house prices than for movie ratings. As this makes comparisons of algorithm performance across datasets difficult, we use the *coefficient of determination* R^2 , which measures the fraction of variance in the data that is explained by the regression model [61]. An R^2 of 1 means that the model explains all variance, while a model that achieves an R^2 of 0 is as good as one that always predicts the mean of the target feature regardless of the input. If R^2 is negative, the model's predictions are even less accurate than always predicting the mean.

Clustering. The mutual information (MI) score describes how much information is shared between two clusters. This metric applies only if the samples are grouped with the same others, regardless of the target labels. We use an adapted version of MI called the *adjusted mutual information* (AMI) [62]. This version corrects the MI score for random choice and normalizes its value to a range between 0 and 1, which is necessary as the MI score tends to increase as the number of clusters increases, regardless of the quality of the clustering produced [63].

6. Results

We discuss our observations grouped by machine learning (ML) task and data quality dimension. For all plots, the horizontal axis indicates the decreasing data quality (training, test, or both) (Section 3) and the vertical axis indicates the increasing ML model performance metric (Section 5.4). The quality of the (unpolluted) baseline dataset is indicated by a dotted vertical line ("Original Data Quality"). For most polluters, the original and the baseline datasets are identical. Otherwise, the quality of the original dataset is indicated by a dashed horizontal line. All values in the plots are an average of five runs for each algorithm per polluted dataset.

For the feature accuracy pollution, we plotted the average of the two metrics $cF Accuracy(d)$ and $nF Accuracy(d)$ described in Section 3.3. Also, due to the definition of consistent representation (Section 3.1), the data quality could increase again (instead of decreasing) with a higher pollution level. Thus, we add the degrees of pollution in the plots for such cases.

We present and discuss the results for each of the scenarios introduced in Section 5.1: Scenario 1 – polluted training set; Scenario 2 – polluted test set; and Scenario 3 – polluted training and test sets. To limit the length of this paper, we include the plots for only one dataset per ML task. The full set of results can be found in the extended work [64].

6.1. Classification

We discuss the effect of degrading the six data quality dimensions of four datasets, namely Credit, Contraceptive, Telco and COVID, on the performance of seven classification algorithms, namely LogR, SVM, DT, GB, KNN, TN, and MLP. We consider MLP in the form of three variants: MLP-1, MLP-5 and MLP-10. We include two baselines trained and tested on clean data, namely a majority-class classifier and a class ratio classifier. The first assigns all test dataset instances to the majority class in the training dataset. The latter selects a label based on the labels' ratios in the training dataset. The target accuracy/balance pollution would shift the class ratios of the training data, which is not reflected in the baseline performance because we report only on clean training and test data. Only for Contraceptive, the classifiers are not binary. Fig. 3 shows the results only for Telco. However, in the description of the result per data quality dimension, respectively data pollution, we also include the results for the other considered datasets.

Although we do not focus on optimizing the prediction accuracy in this study, the results in Fig. 3 in the context of TN mostly show a significantly lower performance than the other considered classification algorithms. We see this effect for the smaller datasets. As a transformer-based model, TN seems to be more data-hungry and requires larger datasets to fully exploit its potential (33 also show this trend in a dedicated experiment). However, for COVID, which is much larger than the other considered datasets, TN achieves similar performance compared to the other classification algorithms.

Consistent representation. Introducing new representations of the categorical values of the training dataset has a limited impact on the performance of the studied classification algorithms on all datasets, as we see in Fig. 3(a) for Telco. However, TN is slightly more erratic than the other algorithms. Considering Telco, performance drops by almost 10 percentage points (%pt) at 20% pollution. Later, at a pollution of at least 45%, the performance increases, then suddenly drops again by around 8%pt. For Scenario 2, we observe a slight and slow decrease in the performance of all algorithms with the decrease in test data consistency. Comparing Figs. 3(b) and 3(c), we can observe a better performance when both training and testing datasets suffer from the same inconsistent representations. In particular, MLP-1 shows a significantly higher susceptibility for pollution in Scenario 2 than MLP-5 and MLP-10. Both variants (MLP-5, MLP-10) show an almost constant performance even after 50% pollution. Also TN shows a constant performance, with only an initial fluctuation in performance up to a pollution of 15%. Considering inconsistent representations with only two representations per original value, we note less resilience by some of the algorithm's and a clear decrease in their performance after polluting 50% of the values in Scenarios 1 and 2.

Completeness. Our intuition was that compromising the completeness of the training data in Scenario 1 leads to classifiers that are increasingly biased towards the imputed placeholder values due to their sheer number. However, the results show a surprisingly limited decline in the F_1 -score, suggesting that the models are affected but not biased. The only exceptions on Telco are the SVM model, which drops drastically in performance once more than half the dataset is polluted, and the GB model, which also shows a significant performance drop after a pollution level of 80%, as Fig. 3(d) shows. In contrast, in Scenario 2 (see Fig. 3(e)), the degradation in prediction performance is faster and ends below the performance of the majority class baseline.

Comparing the results of Scenario 3 to the other scenarios, we notice for all datasets that the risk of classifying incomplete test data seems to be less if the training has already been carried out on incomplete data.

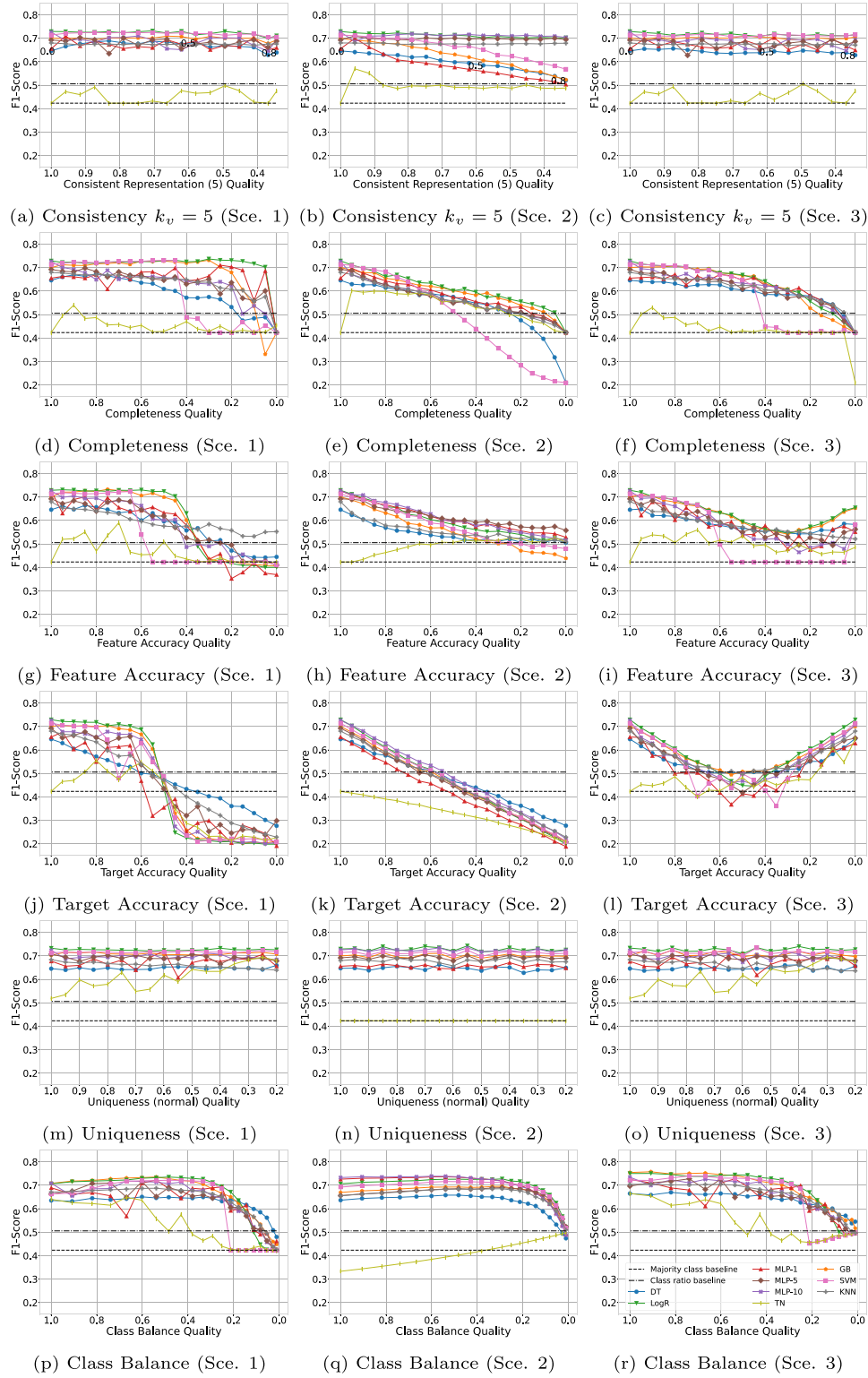


Fig. 3. F_1 -scores of the classification algorithms for Telco dataset.

Feature accuracy. Training the classifiers on noisy data in Scenario 1 has a non-negligible impact on their performance that varies by the dataset. For Contraceptive, the algorithms show a certain robustness until the quality reaches a threshold of 0.8, where the performance degrades more steeply and eventually falls below baseline performance between a quality of 0.4 and 0.2. A similar robustness can be noticed for Credit, except for MLP-1, which performs much worse ($>10\%pt$ drop in F_1 -score) after introducing only a small amount of noise to

the features. The linear models (SVM and LogR) and GB as a tree-based model seem very robust to degrading feature accuracy up to a certain point, where they suddenly lose performance rapidly until they meet the majority class baseline. For the COVID dataset, we observe an initial sharp decline in performance across all algorithms as the feature accuracy decreases, reaching a stable F_1 -score once the quality drops to 0.6. Beyond this point, the F_1 -score remains relatively constant until the quality falls below 0.3.

For Scenario 2 (see Fig. 3(h)), the performances linearly decrease with reduced feature quality of the test data, while staying above baseline performance most of the time. Reducing the quality of the test data to 0.5 causes a drop of about 10%pt in F_1 -score for the linear models. TN shows a slight upward trend in performance, with a 10%pt increase between the original performance and the performance at 100% pollution.

For Scenario 3 (see Fig. 3(i)), we observe a similar behavior as in Scenario 1, but all algorithms show an increase in their performance for low feature accuracy quality.

Target accuracy. The target accuracy is especially relevant in the classification task, as it simulates labeling errors/noise in the data. For Scenario 1, there is almost a linear decline in performance for Contraceptive and Credit in response to decreasing the training dataset target accuracy. However, the performance of the MLP variants drops relatively faster and is more erratic than that of the other algorithms. TN generally follows the trend of the other algorithms but is slightly more erratic considering the Contraceptive dataset. With COVID, all algorithms show a sharp drop in performance at 50% pollution. Telco shows in Fig. 3(j) a similar behavior for all algorithms, except KNN and DT, which steeply decline in performance around a target accuracy of 0.5. Conversely, KNN and DT follow a more linear performance pattern.

For all the datasets and all ML algorithms, we found that once the target accuracy of the training data is equal to or worse than 1 divided by the number of classes, then the prediction performance is below that of the class ratio baseline classifier. In addition, we note that up to 20% of training labels could be flipped without a performance' losses of no more than 10%pt in F_1 -score for most of the algorithms. The performance of all considered MLP variants and SVM also showed a very high variance across the five different seeds, which indicates that they are the most sensitive to incorrectly labeled samples.

The results of Scenario 2 show a linear trend and is consistent across all datasets, with only slight differences in the slopes of the linear trend. 20% more incorrectly labeled samples in the test set leads to a prediction accuracy decrease of up to 10%pt, which shows the importance of carefully labeling the test set. TN is the exception for Credit, with a slightly increasing slope during pollution.

For Scenario 3 (see Fig. 3(l)), we observed a similar behavior of the algorithms as in Scenario 1, with one difference: After polluting 1 divided by the number of classes of the samples both in the training and test data, the F_1 -score starts to increase at different slopes for all algorithms regardless of the dataset. This trend is mainly due to the fact that for datasets with two classes half of the labels have already been flipped at 50% pollution, leading to fewer misclassifications by the algorithms.

Uniqueness. For all datasets, uniqueness does not have much of an impact on the performance of all classifiers in all scenarios, as the Figs. 3(m), 3(n), and 3(o) show. One of the largest drops in the F_1 -score is observed for MLP-1 on Credit for Scenario 1 and 3. This drastic drop in performance is attributed to the size of Credit, only 1000 records, where generalization is already difficult enough, so introducing duplicates gives too much weight to single instances. On the one hand, it is interesting that MLP-1 performance already drops with 5% pollution in Scenarios 1 and 3 on Credit. This suggests that deduplication is an important pre-processing step before training an MLP on a small dataset. On the other hand, the results on the other datasets with linear models suggest that exact duplicates in both training and testing data do not significantly decrease classification performance. However, we see a strong effect on the performance of SVM in the COVID dataset. With a pollution of 20%, there is a sudden change in performance, with the F_1 -score briefly falling by over 15%pt. The influence of duplicates on SVM may be more significant in large datasets, as SVM prioritizes the duplicated points rather than focusing on a broad distribution of points, causing the decision boundary to center more around these points than the overall data.

Target class balance. In Scenarios 1 and 3, once the imbalance affects more than half the samples for the binary classification datasets, all algorithms' performance slowly drops towards the performance of the majority class baseline because they are trained on only a handful of samples from the minority class and therefore have no chance to actually learning the patterns of this class. With COVID, this drop occurs later — when over 90% of the samples are polluted. The performance then drops abruptly by over 10%pt for some algorithms, such as MLP-1 and TN. SVM is an outlier in performance and shows erratic behavior even in the early stages of pollution.

Figs. 3(p) and 3(r) show the described behavior on Telco. All the algorithms behave similarly until this point, with only a few exceptions (mainly SVM and TN). This suggests that the training data does not have to reflect the actual real-world class balance, as long as it is equally or more balanced.

For Scenario 2 (see Fig. 3(q)), we observe that the models are quite robust against the distribution shift, which moves towards a balance in class frequency, and they sometimes even increase their performance. Only TN using Telco and SVM using COVID show a behavior against the trend of the other algorithms.

6.2. Regression

We discuss here the effect of degrading the six data quality dimensions of four datasets, namely: Houses, IMDB, Cars and COVID, on the performance of seven regression algorithms, namely: LR, RR, DT, GB, RF, MLP, TN. Similar to the classification task, we consider MLP in the form of three variants: MLP-1, MLP-5 and MLP-10. As LR and RR differ only in the regularization employed in RR, their performance lines in the result plots often overlap. Fig. 4 shows the results of the experiments using the Houses dataset. We only show R^2 of 0 or larger, since a negative R^2 means that the model is worse than always predicting the mean. Thus, a model in the negative ranges would not be of interest and the resulting vertical axis scale would make it harder to analyze the behavior in the range of interest, 0 to 1. As the results of TN are not visible in the Figures due to TN's low overall performance using small datasets, we additionally included in Fig. 5 the results of the COVID dataset. However, as for classification, we include the findings for the other datasets in the descriptions of the results per data quality dimension.

Consistent representation. An increase of the representations of values in categorical features leads to a decrease in R^2 of all algorithms and datasets. The severity of this decrease depends on the scenario. In Scenario 2 (see Fig. 4(b)), the performance decrease is the largest, especially for RR.

Adding inconsistent representations during training but not during testing, i.e., Scenario 1 (see Fig. 4(a)) has generally a smaller effect on the performance, except for DT. The effect of the pollution is quite low in Scenario 3 for Houses (see Fig. 4(c)), Cars and COVID. For IMDB it is more similar to its Scenario 1. Due to the one-hot encoding of categorical features, LR is affected by the inherently discrete differences. The regularization of RR fixes this extreme behavior. The tree-based methods, RF and GB, are more stable in most cases, along with all considered MLP variants (MLP-1, MLP-5, MLP-10). In Scenarios 1 and 2 for COVID, TN shows sudden unstable behavior at different points. In Scenario 1, TN's is stable over whole pollution increase, whereas in Scenario 2, performance decreases sharply with only a small amount of pollution introduced. The non-linearity of those methods weakens the effect of the inconsistencies.

Completeness. Reducing completeness of a dataset leads to a heavy performance degradation on all algorithms in all scenarios, as shown representatively in Fig. 4 for Houses. The RF and GB performance generally decreases the slowest. These are ensemble methods, which makes them more robust against missing values. Considering Scenarios

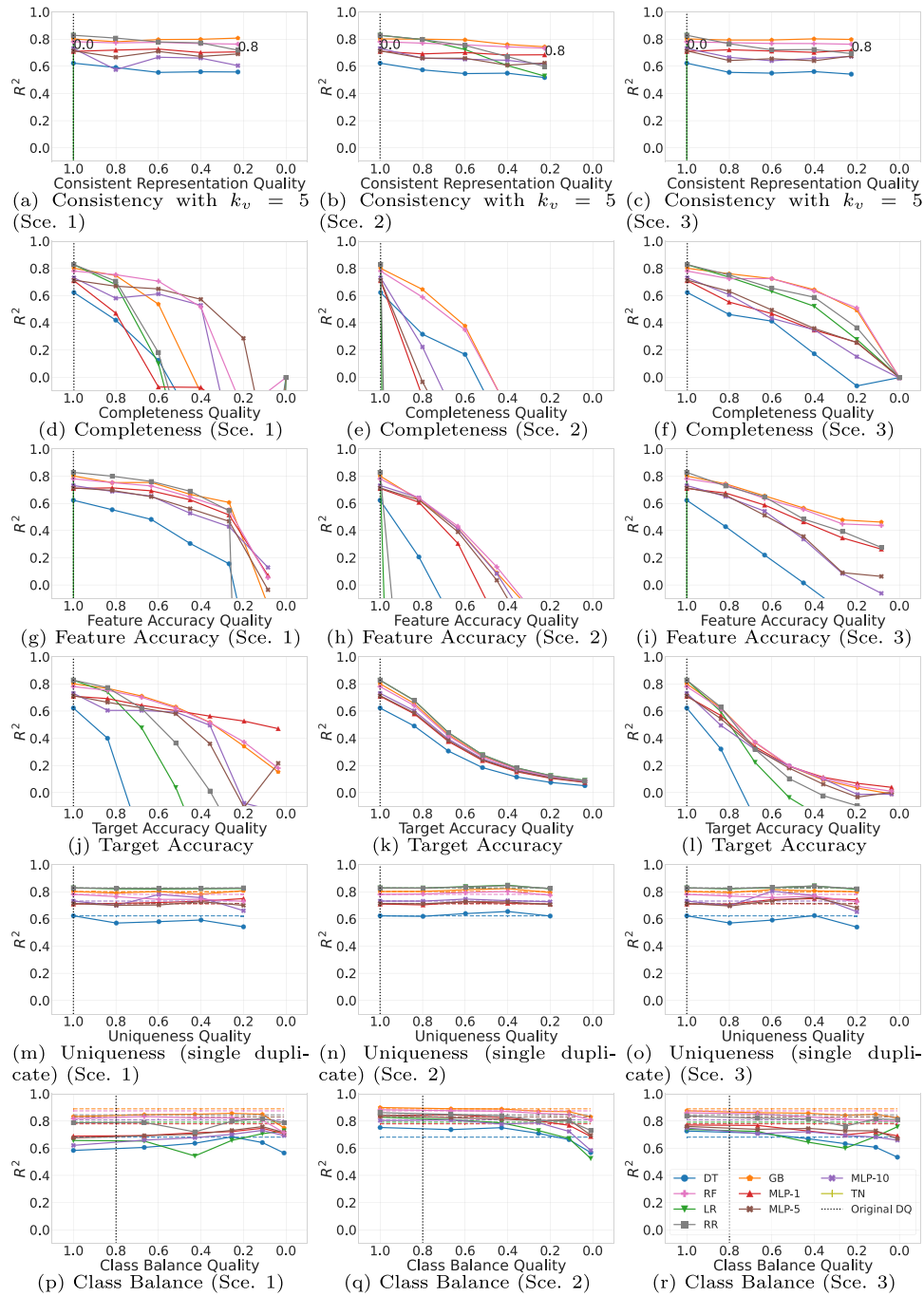


Fig. 4. R^2 of the regression algorithms for Houses dataset.

1 and 3, TN shows the most robust performance using the COVID data (see Fig. 5(a) and 5(c)).

For all datasets, the strongest decrease happens in Scenario 2 (see Fig. 4(e)). LR and RR are the most affected algorithms: the linear relation learned by those algorithms is easier to confuse with the newly inserted placeholders outside the features' domain.

The best performance is achieved in Scenario 3 (see Fig. 4(f)), indicating that models perform better on datasets with missing data if the missing data also exists during the training. The effect of reducing the completeness differs per dataset. For example, the small size of Houses causes the algorithms to overfit and produce better results when training and test sets contain missing values. Whereas larger datasets could reduce this effect, making the amount of non-missing information in the test set the more important factor.

Feature accuracy. Decreasing feature accuracy causes a clear performance degradation of the regression algorithms in all scenarios, as shown representatively in Fig. 4 for Houses.

For Scenario 1, the performance decrease for all considered algorithms, except of DT, is slower in higher quality ranges and faster in lower quality ranges as the differences between training and test set become too large. But also in Scenario 3, the majority of the algorithms, especially RR and GB, perform quite robustly against noise. GB and RR benefit from the normal distribution of the noise. As for completeness, the R^2 degradation is the highest in Scenario 2. In Scenario 3, the algorithms show more linearly decreasing performance, with a stagnation or even improvement in ranges of very low quality (especially for the COVID dataset using TN, GB and RR in Fig. 5(i)). Apparently, the inaccuracies in training and test set align when the pollution is high,

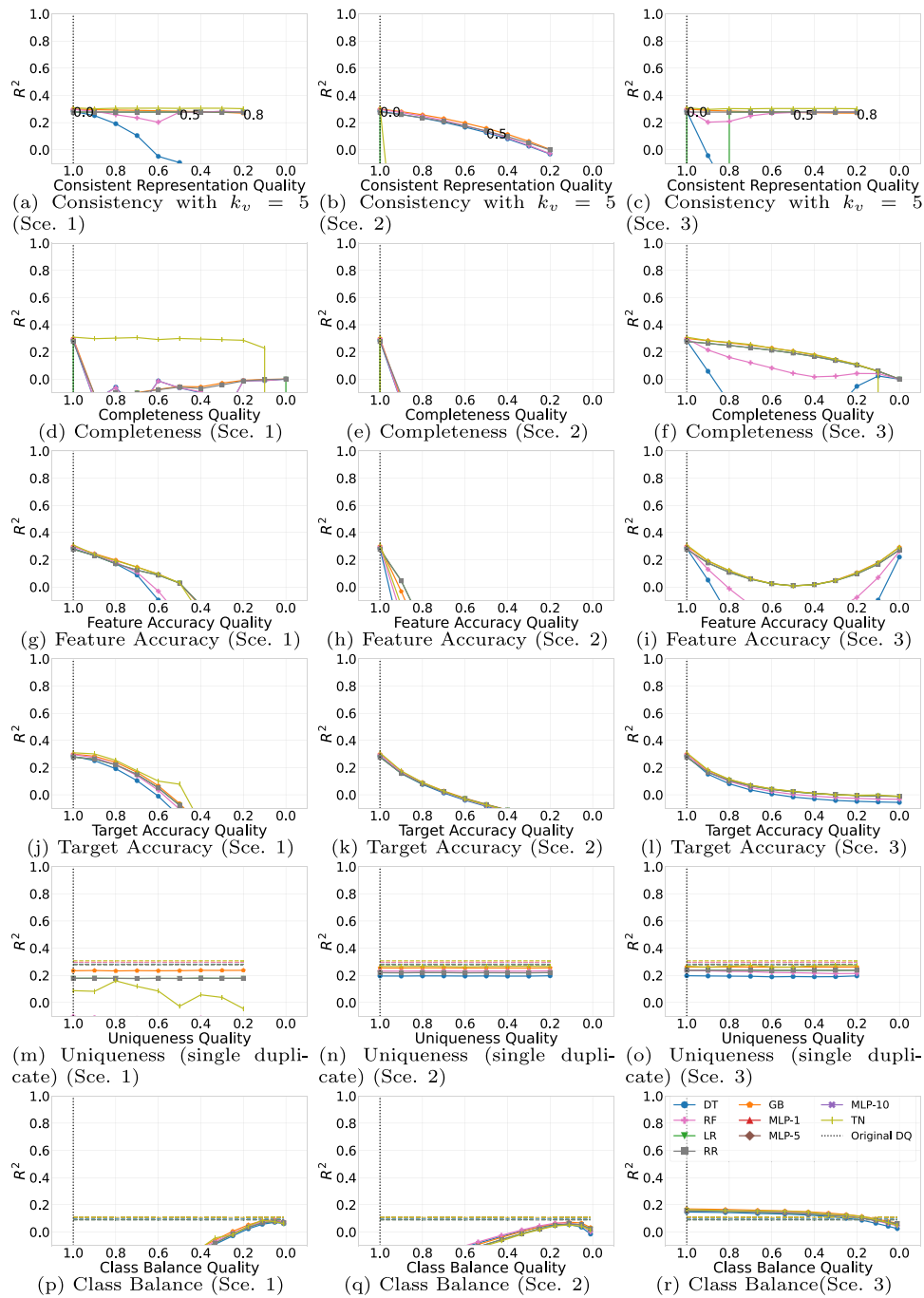


Fig. 5. R^2 of the regression algorithms for COVID dataset.

leading to an increase after a performance minimum. The DT algorithm is more sensitive to feature accuracy drop than the other algorithms in Scenarios 1 and 3. DT's performance decreases especially fast on IMDB and COVID due to the high number of categorical features. All considered MLP variants are generally more robust than DT regarding feature accuracy. There are minimal performance differences between the different MLP variants, except in Scenario 3 in Houses — MLP-5 and MLP-10 have a slightly faster performance drop than MLP-1.

Target accuracy. We observe a strong R^2 degradation in response to decreasing target accuracy for all datasets and in all scenarios, like for feature accuracy and completeness, but at a weaker level.

Scenario 1 shows the strongest degradation with different speeds for different datasets. For Cars, having a low number of features and high

number of records, we observe a heavy performance degradation for RF and DT, whereas the performance of GB and LR/RR stays relatively constant as the noise is normally distributed. In contrast, there is a strong degradation of LR and RR performance for Houses (see Fig. 4(j)) due to the large number of features with low sample size. In general, all MLP variants are quite resilient for low target accuracy. However, similar to feature accuracy, MLP-5 and MLP-10 are less robust than MLP-1 for Houses at a lower target accuracy. Comparing within the LR-based and the tree-based families: RR performs better than LR, and GB outperforms RF and DT.

For all datasets, we can observe that in lower quality ranges, the perceived performance in Scenario 3 (see Fig. 4(l)) decreases slower or even stagnates compared to Scenario 1. Also in Scenario 2 all datasets follow the same trend: The performance declines gradually

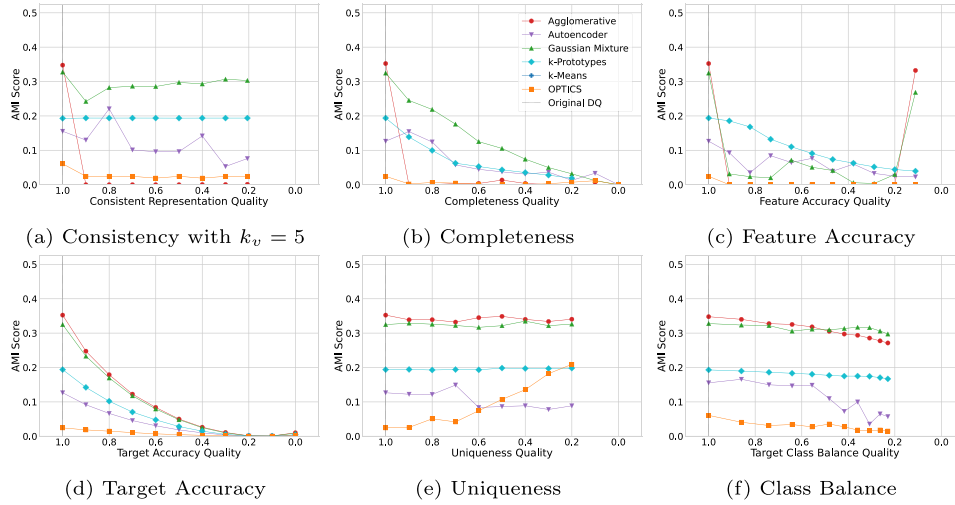


Fig. 6. AMI score of the clustering algorithms for Covertypes.

with the decrease in target accuracy, as the algorithms increasingly fail in predicting the emerging patterns caused by polluting only the test data.

Uniqueness. Decreasing uniqueness does not have a considerable effect on the performance of the algorithms, regardless of the scenario or dataset apart from few exceptions. On the datasets with many samples, such as COVID, we see less effects in the performances, except for TN in Scenario 1, which behaves much more erratic than the simpler models (see Fig. 5(m)). Datasets with a small sample size, like Houses (see Figs. 4(m) and 4(o)), show a slight performance decrease with decreased uniqueness in Scenarios 1 and 3 due to an exaggerated influence of few samples on the training dataset, as they occur several times as a result of duplication.

Comparing the uniqueness with duplicate count sampled by normal distribution and uniqueness with all samples having duplicate count of 1, we do not see different behavior for Cars and IMDB. For Houses and COVID, inserting duplicates following a normal distribution causes a larger degradation of all algorithms' performance, especially for Scenarios 2 and 3. As Cars, IMDB and COVID have fewer features and more samples compared to Houses, this explains the higher resilience of the algorithms.

Target class balance. Similar to the uniqueness dimension, we observe a low effect of the target class imbalance increase in all scenarios on all datasets for imbalance up to 50%, as shown in last row in Fig. 4 for Houses and Fig. 5 for COVID. Two facts can justify this observation: Firstly, regression datasets usually have a continuous target variable, mostly normally distributed and, therefore, inherently unbalanced target variable. Secondly, we had to discretize the target feature and discard resulting classes with very few samples before applying pollution, as described in Section 5.1. Only with a high imbalance above 50%, we start to recognize a degraded performance of all algorithms.

6.3. Clustering

Finally, we discuss in the following section the effect of degrading the six data quality dimensions of four datasets, namely: Bank, Covertypes, Letter and COVID on the performance of five clustering algorithms, namely: Gaussian mixture clustering, k -means/ k -prototypes, agglomerative clustering, OPTICS and autoencoder.

In Fig. 6, we show the results of the experiments using the Covertypes dataset. We only show the y-axis up to an AMI score of 0.5, which is the highest archived score in our experiments.

Here, we start by some general notes and observations which are beneficial for the discussion in the rest of this section. Bank is not originally designed for the clustering task, as it is very low in dimensionality and contains a high amount of duplicated data points. Upon further analysis, we found that some combinations of the two categorical features dominate the data, with the combination of *job = blue-collar* and *marital = married* being found in about 24.8% of samples and the combinations' *technician, married* and *admin., married* together making up another approximately 18.6% of the samples. Therefore, we know that large parts of the data are not distinguishable by anything but the age feature. We question this feature's impact, as it is likely dominated due to the one-hot encoding, which adds more dimensions to the data. As this encodes the categorical values in the data into one dimension per value, it distinguishes the impact of each individual dimension, which disproportionately affects the age feature which is not encoded in this way. Therefore, we are aware that the clustering quality on Bank is expected to be low. We argue that this is not an issue in most cases, as we can still derive some relative changes in clustering quality and argue about the impact of any given quality dimension regarding this dataset. However, we also believe that the findings on Bank are to be taken with a grain of salt and more weight should be put into findings on Covertypes, Letter and COVID.

All algorithms demonstrate significantly greater robustness to lower data quality in the COVID dataset. However, the AMI score is generally much lower compared to the other datasets.

We start by a general observation: We observed that the Gaussian mixture algorithm assumes data that follows a single distribution, i.e., the random peaks and drops in its performance are caused by its sensitivity to the distribution of samples. Bank is drawn from a mixture of Gaussian distributions. Thus, the algorithm performs more robust against low data quality on our other considered datasets (Covertypes, Letter, COVID).

Consistent representation. We do not report on Letter as it has no categorical features. Fig. 6(a) indicates that the performance of the k -means/ k -prototypes and OPTICS algorithms is not affected by the degrading consistency of Covertypes. The same observation applies to Bank. In contrast, the agglomerative algorithm is strongly impaired even with only a small pollution degree. This susceptibility arises because the algorithm uses a Boolean distance matrix to represent differences between values for categorical features rather than a distance matrix with actual distances like for numerical features. This makes it easy to disturb the bottom-up combination process by altering the representation of the values.

For the autoencoder approach, we suspect that its observed random behavior is due to the randomness of the neural network training

and its dependence on the sampled data for each of the five runs, which is amplified in datasets with many features like Coverttype. The Gaussian mixture algorithm's performance slightly decreases on average with decreasing dataset quality. We attribute the unexpected results (initial decrease, then increase in AMI) to the observation made at the beginning of this section.

Completeness. For all datasets, we notice a decline in average AMI for all algorithms as the completeness decreases. Fig. 6(b), shows that this decline is rapid and approaching zero in a negative exponential fashion. Yet, for Coverttype, the Gaussian mixture, autoencoder and k -prototypes algorithms performance degrades almost linearly and in a slightly slower fashion compared to Bank and Letter. This is because of the high number of categorical features that are more resilient against the insertion of default values. As shown in Fig. 6(b), the agglomerative algorithm drops almost to zero at a completeness of 0.9. The added placeholders are not values found in Coverttype. This can create small clusters of outliers far away from the rest of the data points. The effect is exacerbated by the one-hot encoding, where new dimensions are created for the placeholders. As stated before, in COVID, as the largest dataset, the algorithms are generally more robust against any considered data quality issue, including the completeness.

Feature accuracy. For Letter, consisting of only numerical features, we observe a very uniform decrease in the AMI score when decreasing the dataset feature accuracy for almost all algorithms. This is due to the normally distributed noise we apply to numerical features.

In Fig. 6(c), the Gaussian mixture algorithm at the maximum pollution almost reaches the AMI score achieved on the clean dataset: Coverttype consists mainly of binary features that have been completely inverted. The slight difference in performance comes from the ten remaining numerical features. We see a similar behavior in the COVID dataset, including the slight difference in the performance between the fully clean and fully polluted state. The agglomerative algorithm behaves similarly to Gaussian mixture algorithm, but here the drop is again caused by the approach's sensibility to outliers created in the high-dimensional space. Consequently, both the Gaussian mixture and the agglomerative algorithms cannot meaningfully interpret datasets that have mainly binary features and has low feature accuracy. OPTICS behaves similarly on Coverttype and Letter. OPTICS on COVID demonstrates robust performance, showing a stable AMI score regardless of the level of pollution. For all considered datasets the autoencoder and k -Prototypes show an almost linear decrease in AMI score, though this decrease is not as rapid as the rate of quality degradation.

Target accuracy. The pollution of the target accuracy should have no influence on any clustering algorithm, as they are unsupervised: a changed target class does not affect the clustering itself. However, the fact that we can still see a degrading curve in Fig. 6(d) (and also for the other considered datasets) is due to us comparing the clustering result with the polluted target classes and not with the original target classes when calculating the AMI score.

Uniqueness. We used the polluter that inserts duplicates based on a normal distribution. The Gaussian mixture, k -means/ k -prototypes and agglomerative clustering algorithms are not affected by the pollution on Letter and Coverttype as shown in Fig. 6(e). On COVID, Gaussian Mixture shows a slight performance increase of up to 5%pt after pollution. For Coverttype and Letter, the increase in duplicates helps the OPTICS approach to improve its performance: as a density-based clustering algorithm, it can benefit from somewhat evenly inserted duplicates as they increase the overall density of clusters and especially their cores without creating very dense cores consisting of only the same data point many times. The autoencoder approach also enhances its performance above a certain number of duplicates for Letter. In contrast, we cannot see a clear behavior of the autoencoder on Coverttype, which is probably due to the generally random and sample-dependent nature of neural network training. Also, for the COVID dataset, the autoencoder is robust against the inserted duplicates in its performance.

Target class balance. We observe a difference in behavior between Letter with numeric-only features and high-class count and Bank, Coverttype and COVID with fewer classes and features. In Letter, the imbalance in cluster sizes actually slightly improved the AMI score. For example, for OPTICS, we found that the number of clusters being identified on average increased as the dataset grew more imbalanced. For Bank, Coverttype and COVID (see Fig. 6(f)), the k -means/ k -prototypes algorithms' AMI declines slightly and linearly, which can also be observed for the agglomerative clustering on Coverttype. This decline is caused by the inability of these algorithms to identify the shrunken clusters reliably. The irregular behavior of the agglomerative clustering on Bank hints at this algorithm's instability on this particular dataset.

OPTICS and autoencoder show a relatively constant behavior for Bank, COVID, and Coverttype. The Gaussian mixture clustering shows different behavior in Bank, COVID and Coverttype; however, in each case, it does not behave clearly due to the assumption of a specific distribution. Therefore, we believe that altering the samples in the dataset provided to the Gaussian mixture clustering can have positive or negative effects, depending on whether this new sampling more or less resembles the assumed distribution of data.

7. Discussion

In this work, we experimentally studied the impact of six data quality dimensions on 19 ML algorithms from three ML tasks: classification, regression and clustering. We ran many experiments using quality-degraded versions of nine real-world datasets. In addition, we distinguished between three scenarios that a data scientist could face while developing ML pipelines: (1) polluted training set; (2) polluted test set; (3) polluted training and test sets. In the following Section 7.1, we derive high-level trends from the experiments, focusing on the interplay between the ML task and various data quality dimensions. Subsequently, Section 7.2 qualitatively examines selected examples from the experiments where data pollution led to a concrete change in the predictions of the considered ML model.

7.1. Qualitative trends

Table 3 shows a high-level view of our findings, which we summarize along with recommendations in the following paragraphs. The thresholds for categorizing the effects in Table 3 were determined by observing the performance trends, i.e., the average relative change in model performance metrics compared to a baseline established using the clean data, across different levels of data pollution over all datasets and ML models. In particular, we analyzed the performance degradation between using clean data and data polluted to approximately 50% quality. The thresholds were then defined based on these observations:

- Low Effect (✓): If the degradation in performance was less than 5%pt relative to the baseline, we classified the impact as low. Models showing less than a 5%pt drop maintained a performance level that might be acceptable depending on the application context.
- Moderate Effect (○): A performance degradation between 5%pt and 25%pt was considered a moderate effect. This range indicates that the data quality issue has a noticeable impact on model performance, potentially affecting the reliability of the results.
- High Effect (X): Any degradation exceeding 25%pt was deemed a high effect. Such a significant decline suggests that the data quality issue critically undermines the model's effectiveness, necessitating immediate attention and remediation.

Table 3

The effect of data quality dimensions per ML task. ✓: low effect, ○: moderate effect, ×: high effect.

	Consistency	Completeness	Feat.-Accuracy	Tar.-Accuracy	Uniqueness	Class Balance
Classification	✓	×	×	×	✓	○
Regression	✓	×	×	×	✓	○
Clustering	✓	×	×	✓	✓	✓

Example. In classification, regression, and clustering tasks, we noticed that for missing values (especially exceeded 40%pt), the performance degradation on average crossed the 25%pt mark. Therefore, completeness was marked as having a high effect when missing data was substantial.

While the results presented in Section 6 and the following discussion suggest that certain data quality dimensions, such as uniqueness, consistent representation, and target class balance, may not significantly impact the performance of classification, regression, and clustering tasks in some scenarios, they emphasize that the overall goal should always be to maintain high data quality. In critical domains, such as medicine or autonomous systems, even minor data quality issues can have serious consequences. For instance, in medical diagnosis systems, even each misclassification can lead to incorrect diagnoses: a model might misclassify a malignant tumor as benign due to poor target accuracy or missing values in the training data. Thus, relying on lower-quality data should be carefully considered against the potential risks in these settings, where incorrect model outputs could lead to severe outcomes.

Classification. We found that the quality dimensions with the least impact are *uniqueness*, *consistent representation* and *target class balance* as long as the balance is not shifted towards a very extreme case. This suggests that with only a very low loss in classifier performance on tabular data, data scientists can skip pre-processing steps like exact deduplication, unifying inconsistent representations, and carefully balancing the target variable.

For *completeness*, our findings suggest that missing values influence the performance of the classifiers the most when the classifier is not trained on data with missing values. The results also show that having less than 40% missing values in the training phase does not significantly decrease the model's performance, which could be needed to increase robustness if no information is provided on how to impute the test data once the model reaches production.

Reducing the *target accuracy* of the training data results in the degradation of model performance to below a majority classifier performance with target accuracy lower than $1/|classes|$. However, the results suggest that if 80% or more of the data are labeled correctly, one can train classification models on that noisy data without significant performance loss. Our insights also reinforce the importance of labeling test data carefully, as labeling errors could then lead to a significant underestimation of the model's performance.

A very low feature accuracy in the training data can also lead to a strong model performance degradation. However, similar to the target accuracy, there is a certain robustness to such errors, especially for linear models. Yet, the extent of this robustness is dataset-dependent. Reduced feature accuracy during serving time is also problematic but behaves more smoothly and in a linear trend, which makes performance estimation much easier if one can estimate the quality of the serving data. Moreover, even though the performance degrades constantly, it stays above baseline levels most of the time.

Regression. We found that the quality dimensions with the largest impact on regression performance are *completeness*, *feature accuracy* and *target accuracy*. A decrease in quality in one of those dimensions leads to a worse-than-linear decrease in algorithm performance. Furthermore, missing values or inaccurate features in the test data without training the ML algorithm on such kind of data leads to even worse performance. The dimensions *uniqueness* and *target class balance* show little

impact. However, we observed some degradation with small datasets and datasets with many features. We recommend not focusing on these two dimensions when using large training datasets with a few features. *Consistent representation* has an impact as soon as the new representations outweigh the old ones.

For linear regression-based algorithms, ridge regression performs better or is equal to linear regression because regularization helps cope better with polluted data. For tree-based algorithms, random forests always outperform decision trees. The performance of multi-layer perceptron variants highly depend on both the quality dimension and the dataset characteristics. Among all considered algorithms, random forest performs best across most cases and thus is the most robust. One exception is the target accuracy dimension. This insight coincides with the findings by Aleryani et al. investigating imputation methods for classification [65]. They found ensemble learning the best performing approach for imputation when dealing with missing data.

Clustering. We found the quality dimensions of *completeness* and *feature accuracy* to be the most impactful. In both cases, a degradation of the dataset quality led to a decrease in clustering performance, which was linear for datasets with categorical features and seemingly exponential concerning Letter containing only numerical features. The density-based OPTICS algorithm was most affected by *completeness* and *feature accuracy* changes. The *k*-means/*k*-prototypes, agglomerative clustering, and Gaussian mixture clustering generally behaved similarly. The autoencoder approach was least affected by changes in these two quality dimensions.

Interestingly, the Gaussian mixture benefited from the slight noise added to numerical features in the first step of feature accuracy pollution. Therefore, we recommend using this approach when dealing with datasets containing exclusively numerical features if one suspects inaccuracies in their feature values. In the case of mixed-type datasets, we can recommend only the *k*-Prototypes algorithm, as its degradation to initial performance trade-off performs the best in our experiments.

Surprisingly, the *consistent representation*, *target accuracy*, *target class balance*, and *uniqueness* dimensions had a very low impact on the performance of the majority of the examined algorithms. In contrast, adding some duplicate values to a completely duplicate-free dataset, decreasing its uniqueness, significantly improved OPTICS's performance. We recommend refraining from using agglomerative clustering when dealing with a dataset with inconsistent representations for its categorical features, as it could not deal with this pollution. Overall, the *k*-means/*k*-prototypes algorithms, i.e., the centroid-based family, showed the most robustness regarding the six data quality dimensions. Nevertheless, the agglomerative and Gaussian mixture clustering algorithms outperformed the *k*-means/*k*-prototypes approaches on clean data.

7.2. Qualitative examples

In this section, we present selected examples where data pollution influenced the predictions of the considered ML model. Specifically, these examples are records from the test data in which pollution has led to a change in the prediction made for them by the respective ML model. Our focus is on Scenario 3, where both the training and test data are polluted and on the classification task. This section complements our quantitative analysis by illustrating concrete cause-and-effect relationships where pollution changed model predictions. To better understand these relationships, we introduce three cases:

Table 4

Exemplary records from the Telco dataset for selected combinations of ML algorithms and data quality dimensions in the classification task and Scenario 3, considering the cases *bad model*, *bad data* and *lucky model*.

	CombID	Model	Data Quality Dimension	RecordID	Pol. Degree
Bad Model	SC	SVM	Consistency	162	5%
	GC	GB	Completeness	1191	5%
	MT	MLP	Target Accuracy	99	5%
Bad Data	MF	MLP	Feature Accuracy	818	5%
	LT	LogR	Target accuracy	976	5%
	KC	KNN	Completeness	7284	5%
Lucky Model	TC	TN	Consistency	423	5%
	MC	MLP	Completeness	358	5%
	DT	DT	Target accuracy	1351	5%

Bad model When a model is trained on clean data, it makes a correct prediction for a record in the test data. However, if the model is trained on data with $x\%$ pollution, it makes an *incorrect prediction*, even though the *test record* itself remains *unpolluted*.

Bad data When a model is trained on clean data, it makes a *correct prediction* for a record in the test data. However, if the model is trained on the data with $x\%$ pollution, it makes an *incorrect prediction* for a *test record* that is also *polluted*.

Lucky model When a model is trained on clean data, it makes an *incorrect prediction* for a record in the test data. However, when the model is trained on the data with $x\%$ pollution, it (luckily) makes a *correct prediction* for that test record while the *record* itself remains *unpolluted*.

We consider the Telco dataset based on various *combinations* of data quality dimensions and ML algorithms. The test data of Telco consists of 1407 records and 19 features. For the record selection and their assignment to one of the introduced cases, we compare the values of the features and the label before the pollution with those after the pollution. As *uniqueness* and *target class balance* alter the size of the test data during pollution, they are unsuitable for a record wise comparison. Thus, we focus on *consistency*, *completeness*, *feature accuracy*, and *target accuracy* to showcase different behaviors.

For each of the three case, we selected three fixed combinations of ML algorithms and data quality dimensions, while ensuring that each classification algorithm is considered at least once. For each selected combination, we selected a random record among those that first met the criteria for the respective case, i.e., at the lowest pollution level.

Table 4 presents the three cases along with their selected combinations (abbreviated under *CombID*), where each combination has an associated record with the *RecordID* from the test data at the pollution degree (*Pol. degree*). The pollution degree refers to the data as a whole, not specifically to the selected record. That is, a pollution degree of $x\%$ implies that $x\%$ of the data is polluted, but not necessarily the record itself. Note that we added an extra pollution level of 5% for a more fine grained analysis.

In the SC combination, when the SVM algorithm was trained on clean data, it made the correct prediction for this considered record, with the *RecordID* 162. However, when the SVM algorithm is trained on data with 5% pollution, the model misclassifies this record, despite the record itself remaining unpolluted. This highlights a *bad model* case, where the model's prediction is negatively affected by the data pollution elsewhere in the dataset.

Since the test records are actually polluted in the *bad data* case, we also list in Table 5 the original values alongside their respective polluted counterparts only for the polluted features out of the 19 features. For example, for the KC combination at a 5% pollution degree, the KNN model produces an incorrect prediction for the record with

the *RecordID* 284. At this pollution degree, only 2 out of 19 features of the record are polluted. Since KNN relies on feature similarity for classification, the pollution of the features distorts its nearest-neighbor relationships, ultimately leading to misclassification.

Considering the TC combination, TN misclassified the record with the *RecordID* 423 from the test data after training on clean data. However, after introducing inconsistencies in 5% of the training data and training TN on this data, the model correctly classified record 423, although the record itself was not part of the pollution. This represents a *lucky model* case, where the prediction for the considered record was positively influenced by data pollution. However, as demonstrated in our various experiments and summarized in Table 3, this is an exception rather than the rule. In general, data quality issues negatively impact the predictions of the underlying ML model.

7.3. Limitations

The experimental space of our analysis is enormous, leading to some decisions that limit the scope of the evaluation. First, we abstained from optimizing hyperparameters for all algorithms. This lack is intentional, as we are interested in the impact of data quality on the algorithms' performance. However, this omission also means that the reported performance is not necessarily the best for any algorithm — a not unrealistic situation in real/world scenarios. Another “hyperparameter” for the implementation of the completeness polluter is our choice of placeholder values. Comparing different values is a potential area for future work. Next, in the clustering algorithms' evaluation, we implemented the autoencoder using a basic neural network that had not yet been optimized for its particular task. Improvements could include using network components other than linear layers or incorporating the clustering performance on the code space into the loss function. Additionally, we did not account for the information loss caused by encoding high-dimensional data into a two-dimensional embedded space.

8. Conclusion and future work

In the field of artificial intelligence, the importance of data quality is increasingly being recognized, leading to a shift from a model-centric approach to a data-centric perspective.

This study empirically examined the impact of six key data quality dimensions on machine learning performance across three tasks (classification, regression, and clustering) using 19 widely used machine learning algorithms and ten diverse datasets. We conducted the evaluation under various scenarios of high and low training and test data quality. Our results highlight how different data quality issues affect machine learning performance in distinct ways, offering valuable insights for data scientists when developing machine learning pipelines in the presence of data quality issues.

Several avenues for future work emerge. In future work, it is desirable to evaluate the clustering algorithms using not only adjusted mutual information score but also metrics such as the absolute size of overlap between the original and generated data clustering and the average and variance of the cluster sizes in the algorithm output. We also plan to expand our evaluation in two directions: adding more data quality dimensions and conducting a deeper evaluation per machine learning model.

CRediT authorship contribution statement

Sedir Mohammed: Writing – review & editing, Visualization, Validation, Software, Methodology. **Lukas Budach:** Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Data curation. **Moritz Feuerpfel:** Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Data curation. **Nina Ihde:** Writing – original draft, Visualization, Validation, Software,

Table 5

Exemplary records affected by the **bad data** case, including their original and polluted values **only for the polluted features** out of the 19 features per record.

CombID	RecordID	Pol. Degree	Polluted Feature	OriginalValue	Polluted Value
MF	818	5%	tenure	72	71.242689
			PaperlessBill	Yes	No
			PayMethod	Elec check	Credit Card
			MonCharges	110.75	114.472121
			TotCharges	7751.7	7761.316083
LT	976	5%	Churn	No	Yes
KC	284	5%	MultLines	Yes	empty
			TotCharges	2688.45	-1.0

Methodology, Investigation, Data curation. **Andrea Nathansen:** Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Data curation. **Nele Noack:** Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Data curation. **Hendrik Patzlaff:** Writing – original draft, Visualization, Software, Methodology, Data curation, Conceptualization. **Felix Nau-mann:** Writing – review & editing, Supervision, Project administration, Funding acquisition, Conceptualization. **Hazar Harmouch:** Writing – review & editing, Writing – original draft, Validation, Supervision, Project administration, Methodology, Investigation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research was performed partially in the context of the **KITQAR** project, supported in part by Denkfabrik Digitale Arbeitsgemeinschaft im Bundesministerium für Arbeit und Soziales (BMAS).

Data availability

We used open-access data, which we reference in the text. We made the code available on GitHub: <https://github.com/HPI-Information-Systems/DQ4AI>.

References

- [1] C. Gröger, There is no AI without data, *Commun. ACM* 64 (11) (2021) 98–108, <http://dx.doi.org/10.1145/3448247>.
- [2] E. Breck, N. Polyzotis, S. Roy, S. Whang, M. Zinkevich, Data validation for machine learning, in: *Proceedings of Machine Learning and Systems (MLSys)*, mlsys.org, 2019, URL https://proceedings.mlsys.org/paper_files/paper/2019/hash/928f1160e52192e3e0017fb63ab65391-Abstract.html.
- [3] C. Ré, The road to software 2.0 or data-centric AI, 2021, URL <https://hazyresearch.stanford.edu/data-centric-ai>.
- [4] R.Y. Wang, D.M. Strong, Beyond accuracy: What data quality means to data consumers, *J. Manage. Inf. Syst.* 12 (4) (1996) 5–33, URL <http://www.jmis-web.org/articles/1002>.
- [5] C. Batini, M. Scannapieco, *Data Quality: Concepts, Methods and Techniques*, Springer Verlag, Berlin – Heidelberg – New York, 2006.
- [6] V. Gudivada, A. Apon, J. Ding, Data quality considerations for big data and machine learning: Going beyond data cleaning and transformations, *Int. J. Adv. Softw.* 10 (1) (2017) 1–20.
- [7] N. Polyzotis, S. Roy, S.E. Whang, M. Zinkevich, Data lifecycle challenges in production machine learning: A survey, *SIGMOD Rec.* 47 (2) (2018) 17–28, <http://dx.doi.org/10.1145/3299887.3299891>.
- [8] C. Renggli, L. Rimanic, N.M. Gürel, B. Karlas, W. Wu, C. Zhang, A data quality-driven view of MLOps, *IEEE Data Eng. Bull.* 44 (1) (2021) 11–23, URL <http://sites.computer.org/debull/A21mar/p11.pdf>.
- [9] S. Schelter, F. Bießmann, T. Januschowski, D. Salinas, S. Seufert, G. Szarvas, On challenges in machine learning model management, *IEEE Data Eng. Bull.* 41 (4) (2018) 5–15, URL <http://sites.computer.org/debull/A18dec/p5.pdf>.
- [10] N. Gupta, H. Patel, S. Afzal, N. Panwar, R.S. Mittal, S.C. Guttula, A. Jain, L. Nagalapatti, S. Mehta, S. Hans, P. Lohia, A. Aggarwal, D. Saha, Data quality toolkit: Automatic assessment of data quality and remediation for machine learning datasets, *CoRR* (2021) URL [arXiv:2108.05935](https://arxiv.org/abs/2108.05935).
- [11] S. Schelter, D. Lange, P. Schmidt, M. Celikel, F. Bießmann, A. Grafberger, Automating large-scale data quality verification, *PVLDB* 11 (12) (2018) 1781–1794, <http://dx.doi.org/10.14778/3229863.3229867>, URL <http://www.vldb.org/pvldb/vol11/p1781-schelter.pdf>.
- [12] T. Gebru, J. Morgenstern, B. Vecchione, J.W. Vaughan, H.M. Wallach, H.D. III, K. Crawford, Datasheets for datasets, *Commun. ACM* 64 (12) (2021) 86–92, <http://dx.doi.org/10.1145/3458723>.
- [13] J. Stoyanovich, B. Howe, Nutritional labels for data and models, *IEEE Data Eng. Bull.* 42 (3) (2019) 13–23, URL <http://sites.computer.org/debull/A19sept/p13.pdf>.
- [14] J. Tagliabue, V. Tuulos, C. Greco, V. Dave, DAG card is the new model card, *CoRR* (2021) 1–6, URL [arXiv:2110.13601](https://arxiv.org/abs/2110.13601).
- [15] P. Li, X. Rao, J. Blase, Y. Zhang, X. Chu, C. Zhang, CleanML: A study for evaluating the impact of data cleaning on ML classification tasks, in: *Proceedings of the International Conference on Data Engineering, ICDE, 2021*, pp. 13–24, <http://dx.doi.org/10.1109/ICDE51399.2021.00009>.
- [16] D. Foroni, M. Lissandrini, Y. Velegrakis, Estimating the extent of the effects of data quality through observations, in: *Proceedings of the International Conference on Data Engineering, ICDE, 2021*, pp. 1913–1918, <http://dx.doi.org/10.1109/ICDE51399.2021.00176>.
- [17] F. Neutatz, B. Chen, Y. Alkhatib, J. Ye, Z. Abedjan, Data cleaning and automl: Would an optimizer choose to clean? *Datenbank Spektrum* 22 (2) (2022) 121–130, <http://dx.doi.org/10.1007/s13222-022-00413-2>.
- [18] V. Shah, T. Parashos, A. Kumar, How do categorical duplicates affect ML? A new benchmark and empirical analyses, *PVLDB* 17 (6) (2024) 1391–1404, <http://dx.doi.org/10.14778/3648160.3648178>.
- [19] B. Frénay, M. Verleysen, Classification in the presence of label noise: A survey, *IEEE Trans. Neural Networks Learn. Syst.* 25 (5) (2014) 845–869, <http://dx.doi.org/10.1109/TNNLS.2013.2292894>.
- [20] S. Schelter, T. Rukat, F. Bießmann, Learning to validate the predictions of black box classifiers on unseen data, in: *Proceedings of the International Conference on Management of Data, SIGMOD, ACM, 2020*, pp. 1289–1299, <http://dx.doi.org/10.1145/3318464.3380604>.
- [21] F. Biessmann, J. Golebiowski, T. Rukat, D. Lange, P. Schmidt, Automated data validation in machine learning systems, *IEEE Data Eng. Bull.* 44 (1) (2021) 51–65, URL <http://sites.computer.org/debull/A21mar/p51.pdf>.
- [22] S. Schelter, T. Rukat, F. Biessmann, JENGA – a framework to study the impact of data errors on the predictions of machine learning models, in: *Proceedings of the International Conference on Extending Database Technology, EDBT, OpenProceedings.org, 2021*, pp. 529–534, <http://dx.doi.org/10.5441/002/edbt.2021.63>.
- [23] M. Feurer, A. Klein, K. Eggenberger, J.T. Springenberg, M. Blum, F. Hutter, Efficient and robust automated machine learning, in: *Advances in Neural Information Processing Systems, NeurIPS, 2015*, pp. 2962–2970, URL <https://proceedings.neurips.cc/paper/2015/hash/11d0e6287202fcd83f79975ec59a3a6-Abstract.html>.
- [24] F. Neutatz, B. Chen, Z. Abedjan, E. Wu, From cleaning before ML to cleaning for ML, *IEEE Data Eng. Bull.* 44 (1) (2021) 24–41, URL <http://sites.computer.org/debull/A21mar/p24.pdf>.
- [25] R.J.A. Little, D.B. Rubin, *Statistical Analysis with Missing Data*, second ed., Wiley, 2014, <http://dx.doi.org/10.1002/9781119013563>.
- [26] C.N.S. Kumar, K.N. Rao, A. Govardhan, N. Sandhya, Subset k-means approach for handling imbalanced-distributed data, in: *Emerging ICT for Bridging the Future- Proceedings of the Annual Convention of the Computer Society of India CSI Volume 2*, Springer, 2015, pp. 497–508.
- [27] P. McCullagh, J.A. Nelder, *Generalized Linear Models*, Springer, 1989, <http://dx.doi.org/10.1007/978-1-4899-3242-6>.
- [28] C. Cortes, V. Vapnik, Support-vector networks, *Mach. Learn.* 20 (3) (1995) 273–297.

- [29] L. Breiman, J.H. Friedman, R.A. Olshen, C.J. Stone, *Classification and Regression Trees*, Wadsworth, 1984.
- [30] J.H. Friedman, Greedy function approximation: A gradient boosting machine, *Ann. Statist.* 29 (5) (2001) 1189–1232, <http://dx.doi.org/10.1214/aos/1013203451>.
- [31] N.S. Altman, An introduction to kernel and nearest-neighbor nonparametric regression, *Amer. Statist.* 46 (3) (1992) 175–185, <http://dx.doi.org/10.1080/00031305.1992.10475879>.
- [32] D.F. Specht, A general regression neural network, *IEEE Trans. Neural Netw.* 2 (6) (1991) 568–578, <http://dx.doi.org/10.1109/72.97934>.
- [33] S.Ö. Arik, T. Pfister, TabNet: Attentive interpretable tabular learning, in: *Proceedings of the National Conference on Artificial Intelligence*, AAAI, AAAI Press, 2021, pp. 6679–6687, <http://dx.doi.org/10.1609/AAAI.V35I8.16826>.
- [34] D.C. Montgomery, E.A. Peck, G.G. Vining, *Introduction to Linear Regression Analysis*, John Wiley & Sons, 2021.
- [35] A.E. Hoerl, R.W. Kennard, Ridge regression: Biased estimation for nonorthogonal problems, *Technometrics* 42 (1) (2000) 80–86, <http://dx.doi.org/10.1080/00401706.2000.10485983>.
- [36] L. Breiman, Random forests, *Mach. Learn.* 45 (1) (2001) 5–32, <http://dx.doi.org/10.1023/A:1010933404324>.
- [37] L. Rokach, O. Maimon, *Clustering methods*, in: *The Data Mining and Knowledge Discovery Handbook*, Springer, 2005, pp. 321–352.
- [38] D.A. Reynolds, Gaussian mixture models, in: *Encyclopedia of Biometrics*, Springer US, 2009, pp. 659–663, http://dx.doi.org/10.1007/978-0-387-73003-5_196.
- [39] K.P. Sinaga, M. Yang, Unsupervised k-means clustering algorithm, *IEEE Access* 8 (2020) 80716–80727, <http://dx.doi.org/10.1109/ACCESS.2020.2988796>.
- [40] J. Ji, T. Bai, C. Zhou, C. Ma, Z. Wang, An improved k-prototypes clustering algorithm for mixed numeric and categorical data, *Neurocomputing* 120 (2013) 590–596, <http://dx.doi.org/10.1016/J.NEUCOM.2013.04.011>.
- [41] W.H.E. Day, H. Edelsbrunner, Efficient algorithms for agglomerative hierarchical clustering methods, *J. Classification* 1 (1) (1984) 7–24, <http://dx.doi.org/10.1007/BF01890115>.
- [42] M. Ankerst, M.M. Breunig, H. Kriegel, J. Sander, OPTICS: Ordering points to identify the clustering structure, *Proc. Int. Conf. Manag. Data (SIGMOD)* 28 (2) (1999) 49–60, <http://dx.doi.org/10.1145/304182.304187>.
- [43] C. Song, F. Liu, Y. Huang, L. Wang, T. Tan, Auto-encoder based data clustering, in: *Iberoamerican Congress on Pattern Recognition*, 8258, Springer Berlin Heidelberg, 2013, pp. 117–124, http://dx.doi.org/10.1007/978-3-642-41822-8_15.
- [44] IBM, Telco customer churn dataset, 2018, URL <https://www.kaggle.com/blatchar/telco-customer-churn>, visited 2024-11-11.
- [45] U. Grömping, South German Credit Data: Correcting a Widely Used Data Set, Technical Report, Beuth Hochschule für Technik Berlin, 2019, URL http://www1.beuth-hochschule.de/FB_II/reports/Report-2019-004.pdf.
- [46] T.-S. Lim, Contraceptive method choice, 1999, <http://dx.doi.org/10.24432/C59W2D>, URL <https://archive.ics.uci.edu/dataset/30/contraceptive+method+choice>, visited 2024-11-11.
- [47] M. government, Información referente a casos COVID-19 en México, 2020, URL <https://datos.gob.mx/busca/dataset/informacion-referente-a-casos-covid-19-en-mexico>, visited 2024-10-09.
- [48] Mexican government, COVID-19 dataset, 2020, URL <https://www.kaggle.com/datasets/meirizri/covid19-dataset>, visited 2024-10-09.
- [49] D.D. Cock, Ames, Iowa: Alternative to the Boston housing data as an end of semester regression project, *J. Stat. Educ.* 19 (3) (2011) <http://dx.doi.org/10.1080/10691898.2011.11889627>.
- [50] Kaggle, House prices – advanced regression techniques, 2016, URL <https://kaggle.com/competitions/house-prices-advanced-regression-techniques>.
- [51] M. Ramadan, IMDb most popular films and series, 2021, URL <https://www.kaggle.com/mazenramadan/imdb-most-popular-films-and-series/version/3>, visited 2022-03-11.
- [52] Aditya, 100,000 UK used car data set, 2020, URL <https://www.kaggle.com/adityadesai13/used-car-dataset-ford-and-mercedes/version/3>, visited 2022-03-11.
- [53] D. Slate, Letter recognition dataset, 1991, <http://dx.doi.org/10.24432/C5ZP40>, URL <https://archive.ics.uci.edu/ml/datasets/letter+recognition>, visited 2024-11-11.
- [54] D. Dua, C. Graff, UCI machine learning repository, 2017, URL <http://archive.ics.uci.edu/ml>, visited 2022-03-11.
- [55] S. Moro, P. Cortez, P. Rita, Bank marketing, 2014, <http://dx.doi.org/10.24432/C5K306>, URL <https://archive.ics.uci.edu/ml/datasets/bank+marketing>, visited 2024-13-11.
- [56] S. Moro, P. Cortez, P. Rita, A data-driven approach to predict the success of bank telemarketing, *Decis. Support Syst.* 62 (2014) 22–31.
- [57] J. Blackard, Covertypes, 1998, <http://dx.doi.org/10.24432/C50K5N>, URL <https://archive.ics.uci.edu/dataset/31/covertime>, visited 2024-13-11.
- [58] J.A. Blackard, D.J. Dean, Comparative accuracies of artificial neural networks and discriminant analysis in predicting forest cover types from cartographic variables, *Comput. Electron. Agric.* 24 (3) (1999) 131–151, [http://dx.doi.org/10.1016/S0168-1699\(99\)00046-0](http://dx.doi.org/10.1016/S0168-1699(99)00046-0).
- [59] J.C. Gower, A general coefficient of similarity and some of its properties, *Biometrics* 27 (04) (1971) 857–871.
- [60] D.P. Kingma, J. Ba, Adam: A method for stochastic optimization, in: *International Conference on Learning Representations, ICLR*, 2015, URL <http://arxiv.org/abs/1412.6980>.
- [61] C. Lewis-Beck, M. Lewis-Beck, *Applied Regression: An Introduction*, vol. 22, Sage publications, 2015.
- [62] X.V. Nguyen, J. Epps, J. Bailey, Information theoretic measures for clusterings comparison: is a correction for chance necessary? in: *Proceedings of the International Conference on Machine Learning, ICML*, 382, ACM, 2009, pp. 1073–1080, <http://dx.doi.org/10.1145/1553374.1553511>.
- [63] X.V. Nguyen, J. Epps, J. Bailey, Information theoretic measures for clusterings comparison: Variants, properties, normalization and correction for chance, *J. Mach. Learn. Res.* 11 (2010) 2837–2854, <http://dx.doi.org/10.5555/1756006.1953024>, URL <https://dl.acm.org/doi/10.5555/1756006.1953024>.
- [64] L. Budach, M. Feuerpfeil, N. Ihde, A. Nathansen, N.S. Noack, H. Patzlaff, F. Naumann, H. Harmouch, The effects of data quality on ML-model performance, *CoRR abs/2207.14529* (2024) <http://dx.doi.org/10.48550/arXiv.2207.14529>, arXiv:2207.14529.
- [65] A. Aleryani, W. Wang, B. de la Iglesia, Multiple imputation ensembles (MIE) for dealing with missing data, *SN Comput. Sci.* 1 (3) (2020) 1–20, <http://dx.doi.org/10.1007/S42979-020-00131-0>.